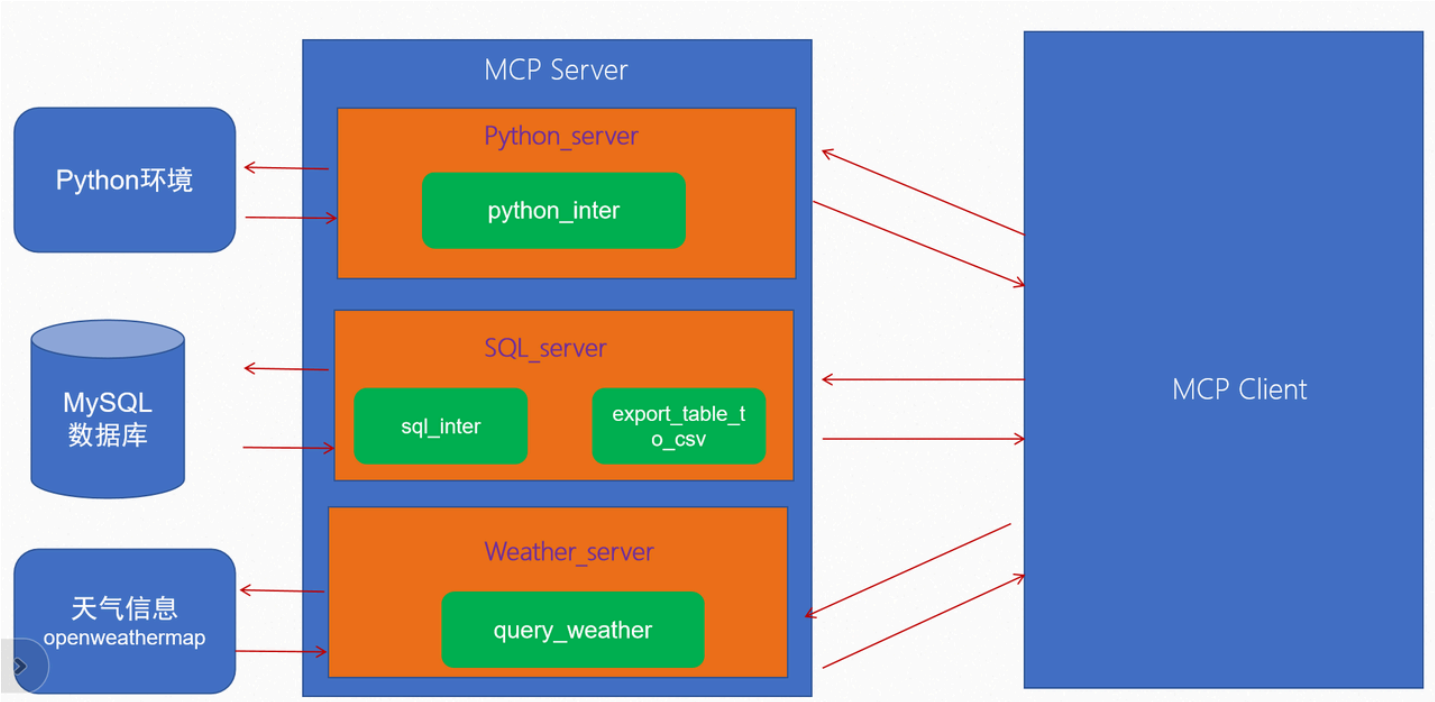


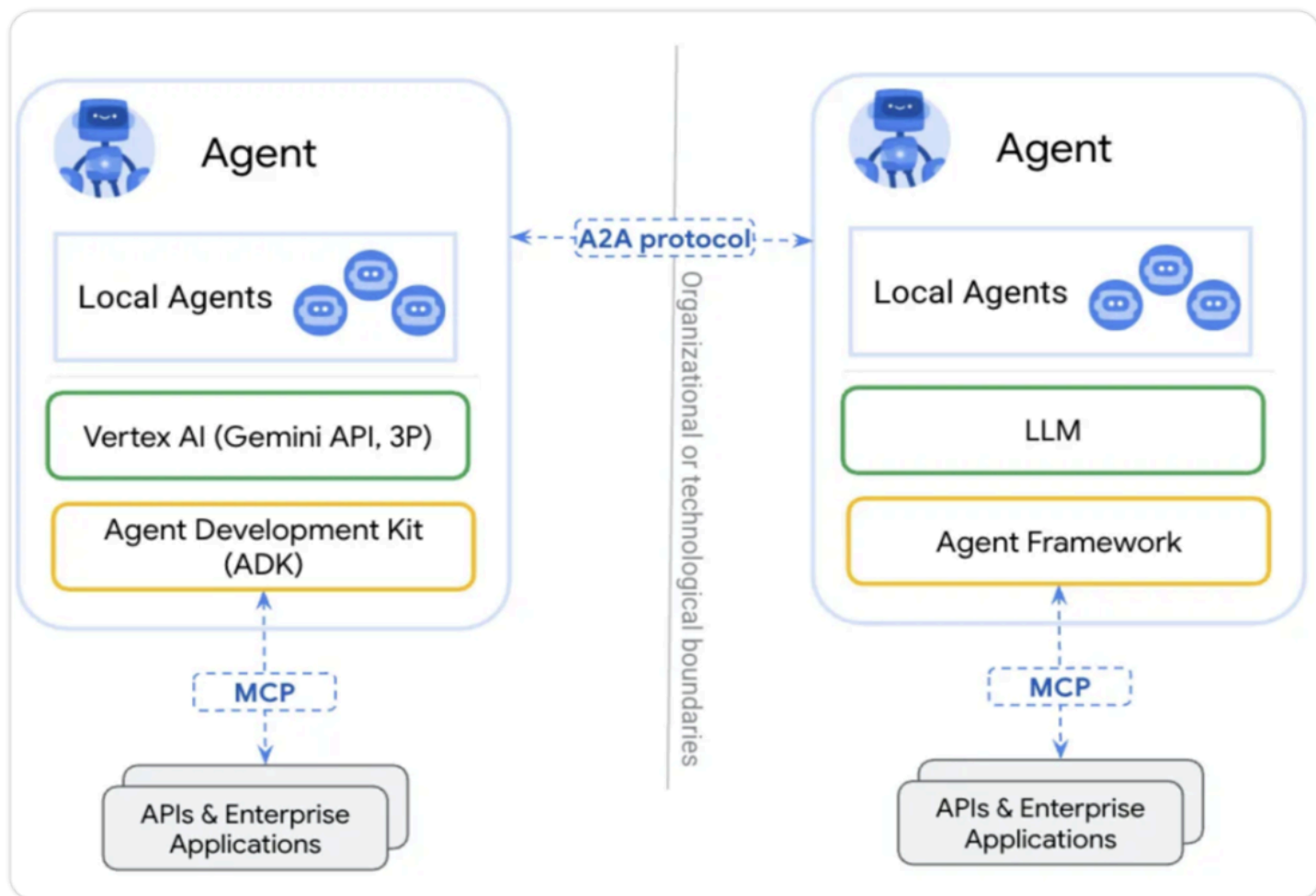
企业级多智能体工程落地协议 A2A

1. 什么是 A2A 协议？

1.1 为什么会出现 A2A 协议？



谷歌开源了首个标准智能体交互协议——Agent2Agent Protocol（简称A2A）

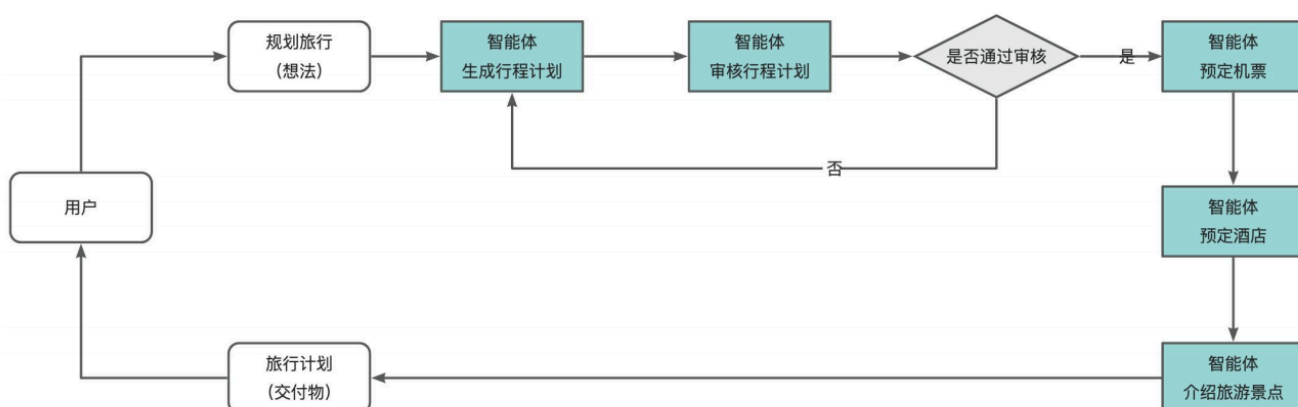


gent2Agent (A2A) 协议是一个开放标准：

- 👍 解决Agent 与Agent之间的通信和调用规则
- 让所有的Agent智能体都遵守这个规则
- 将所有的Agent 连接起来协同工作。

1.2 A2A 协议解决了什么问题？

旅行APP





智能体（AI Agent）处理不同的事情

多个智能体需要通过消息传递和调用完成整件事情

智能体之间存在一次或者多次沟通协作

生成智能体的团队有可能不同。

智能体用到的技术有可能不同。

所以需要有一个协议（A2A）解决上述问题。

1.3 A2A 协议是如何解决问题的？

为“不透明”的（黑盒）代理系统提供了一种标准化的互动方式。它定义了：

1. 通用的传输和格式：JSON-RPC 2.0 通过 HTTP(S)用于消息的结构和传输方式。

想象每个 AI 代理就像一个只会说方言的小店老板。以前他们互相听不懂，没法合作。A2A 协议规定：大家都用 JSON-RPC 2.0 这种标准“普通话”写消息，并且通过 HTTP/HTTPS（就是上网常用的协议）来发送。这样一来，不管谁写的代理，只要遵守这个规则，就能互相“打电话”、“发短信”，沟通无障碍。

2. 发现机制（代理卡）：代理如何宣传其功能并被其他代理发现。打标签。

每个代理可以发一张“名片”（叫“代理卡”），上面写着：“我会翻译”“我能查天气”“我能处理表格”……还打上标签，比如 #翻译 #数据分析。别的代理看到这张名片，就知道“哦，有事可以找他帮忙”。这就解决了“我不知道谁能帮我”的问题。

3. 任务管理工作流程：协作任务如何启动、进行和完成。这包括支持可能是长时间运行或需要多次交互的任务。

有时候一个任务不是一句话就能搞定的，比如“帮我订机票+酒店+安排接送”，可能要来回问好几次、花好几天。A2A 协议规定了怎么开始任务、中间怎么传递进度、最后怎么确认完成，甚至支持“你先去做，我等你结果”这种异步协作。这样多个代理配合起来就像一支有纪律的团队，不会乱套。

4. 支持各种数据模式：代理如何交换不仅仅是文本，还有文件、结构化数据（如表单）以及可能的其他丰富媒体。

以前很多系统只能传文字，但现实中我们经常需要传 Excel 表格、PDF 文件、图片等。A2A 协议允许代理之间交换多种类型的数据，比如上传一个 CSV 文件让另一个代理分析，或者返回一张生成的图表。这让合作更实用、更贴近真实工作场景。

5. 安全性和异步性的核心原则：安全通信和处理可能需要较长时间或涉及人机协作流程的任务的指导方针。

- 安全性：所有通信都建议走 HTTPS，防止被偷听或篡改；还可以加身份验证，确保“找对人、不被冒充”。
- 异步性：有些任务很慢（比如训练模型、人工审核），不能一直等着。协议支持“你先忙，好了通知我”，这样系统不会卡住，也能和人类协作（比如等人审批后再继续）。

总结一下：

 A2A 协议就像给一群“黑盒子”AI代理制定了一套合作公约——统一语言、亮明技能、规范流程、丰富工具、保障安全。从此它们不再是孤岛，而是能组队打怪的“智能联盟”。

2. A2A 协议是如何设计的？

2.1 A2A 协议设计的关键原则

A2A 的发展受到几个核心原则的指导：

1. 简单性：在可能的情况下利用现有且广为人知的标准，如 HTTP、JSON

RPC 和 Server-Sent Events (SSE)，而不是重复造轮子。

A2A 不搞花里胡哨的新技术，而是直接用大家早就熟悉、广泛支持的标准，比如：

- HTTP/HTTPS（就是你上网用的协议），
- JSON-RPC（一种结构清晰、好读好写的通信格式），
- SSE (Server-Sent Events)（一种服务器主动推送消息给客户端的方式）。

打个比方：就像超市只提供大、中、小三种购物袋——简单、够用、谁都会用。不用每个顾客都自带袋子，也不用超市发明一种新袋子。这样开发起来快，维护也省心。

2. 企业就绪性：通过与标准网络实践保持一致，从一开始就解决关键的企业需求，如身份验证、授权、安全性、隐私、追踪和监控。

很多新技术在实验室很好用，但一进大公司就“水土不服”。A2A 从设计之初就想着企业实际需求：

- 谁能调用这个代理？（身份验证）
- 他有没有权限做这件事？（授权）
- 数据会不会被偷看或篡改？（安全性 & 隐私）
- 出问题了能不能查日志？（追踪 & 监控）

就像开一家正规餐厅，不是光会炒菜就行，还得有营业执照、消防通道、监控摄像头——A2A 把这些“合规装备”都提前配齐了。

3. 优先考虑异步：通过流式传输和推送通知等机制，本地支持长时间运行的任务和代理或用户可能不断连接的场景。代理/智能体之间有可能是网络调用，内部逻辑处理时长不可控。

现实中很多任务很慢：比如让一个代理生成报告、处理视频，或者等人审批。如果每次都“你发请求→我死等回复”，系统就会卡住。

A2A 更推崇 异步协作：你发个任务给我，我可以边做边通过“流式传输”或“推送通知”告诉你进度（比如“50%完成了”“出错了”）。你不用一直挂着，可以去做别的事。这特别适合网络不稳定、任务耗时长、或者需要人机混合协作的场景。

4. 模态不可知：允许代理使用各种内容类型进行通信，实现丰富灵活的交互，

超越纯文本。比如文字、图片、音频、视频等。

传统聊天机器人只能处理文字，但真实世界需要更多：图片、语音、表格、视频、PDF……

A2A 的设计是“模态不可知”的，意思是：它不限定你必须用哪种内容。你想传音频？行。想回一张图表？没问题。代理之间可以像人类一样，用最合适的“媒介”交流，而不是被文字绑死。

就像两个人合作，有时候发个截图比写一百句话更清楚——A2A 支持这种灵活沟通。

5. 不透明执行：实现协作，无需要求代理公开其内部逻辑、内存或专有工具。

代理基于声明的能力和交换的上下文进行交互，保护知识产权并增强安全性。

每个 AI 代理可能用了自家的秘密算法、私有工具或敏感数据。A2A 不要求你把“内部怎么运作的”公开出来（不像开源那样必须亮代码）。

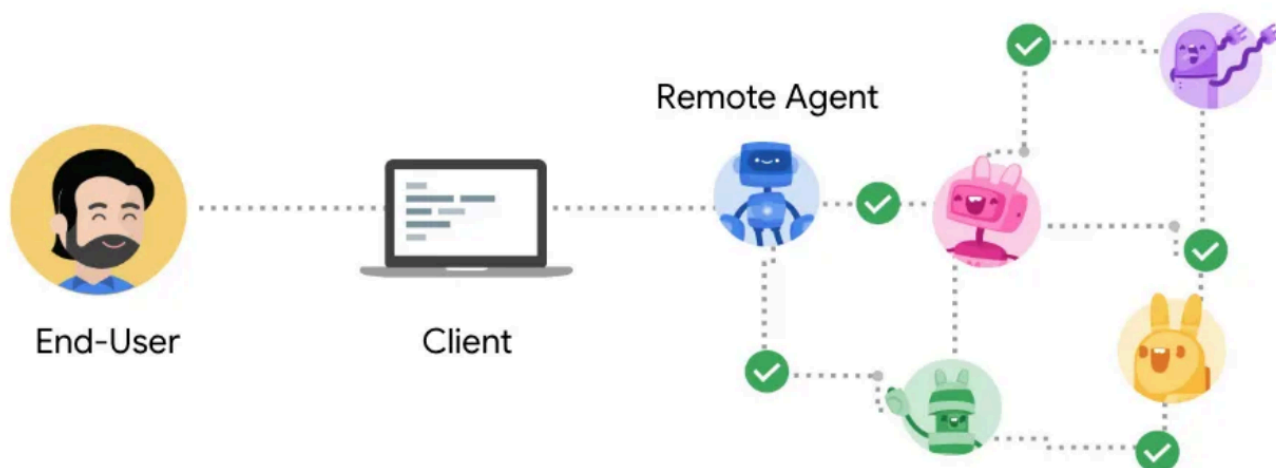
你只需要告诉别人：“我能做翻译”“我能分析销售数据”，然后按约定格式交换信息就行。具体你怎么做的？那是你的事。这样既保护了知识产权，又避免泄露商业机密，还能防止别人钻空子攻击你的内部逻辑。

就像你叫外卖：你只关心饭好不好吃、送没送到，不需要知道厨师用哪口锅、放了几勺盐。

✅ 总结一下：

👍 A2A 的五大原则就是为了让 AI 代理之间简单、安全、高效、灵活、隐私友好地协作——用现成工具、满足企业要求、支持长时间任务、兼容各种内容、还不暴露内部秘密。一句话：让黑盒代理也能组队打配合，而且打得专业又安心。

2.2 A2A 协议核心概念之参与者



1. 用户：发起需要代理协助的请求或目标的最终用户（人类或自动化服务）。
2. A2A 客户端（客户端代理）：一个应用程序、服务或另一个 AI 代理，代表用户请求远程代理执行操作或获取信息。客户端使用 A2A 协议发起通信。
3. A2A 服务器（远程代理）：一个 AI 代理或代理系统，提供实现 A2A 协议的 HTTP 端点。它接收来自客户端的请求，处理任务，并返回结果或状态更新。从客户端的角度来看，远程代理作为一个“不透明”系统运行，这意味着客户端不需要了解其内部工作、内存或工具。

✅ 一句话总结：

👍 用户提需求 → 客户端代理去“找人” → 远程代理默默干活并返回结果，全程不用知道对方肚子里怎么想的。

就像你叫外卖：你（用户）下单，App（客户端）联系餐厅（远程代理），餐厅做好送来——你不需要知道厨房怎么炒菜。

2.3 A2A 协议核心概念之基本通信要素

1. 代理卡片: 让其他的代理找到自己

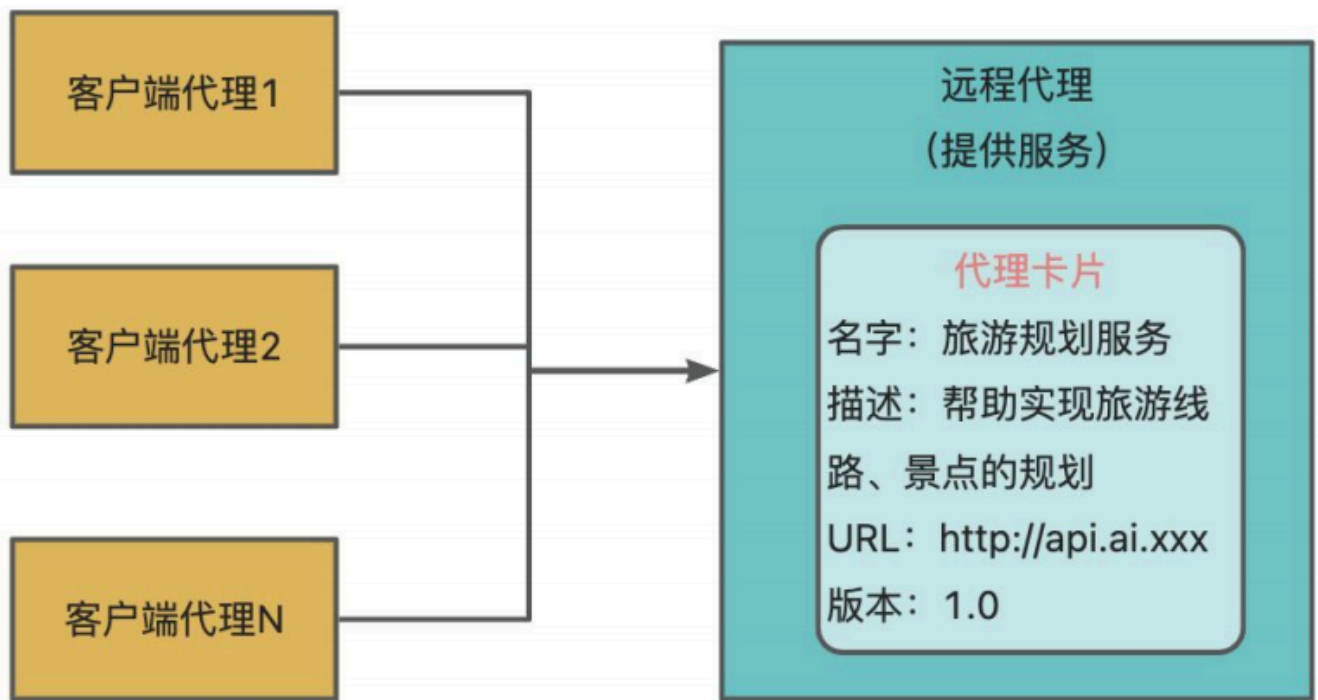
- 一个 JSON 元数据文档，通常可以在一个众所周知的 URL（例如，`/.well-known/agent.json`）上找到，描述了一个 A2A 服务器。
- 它详细描述了代理的身份（名称、描述）、服务端点 URL、版本、支持的 A2A 能力（如流式传输或推送通知）、它提供的具体技能、默认的输入/输出模式以及认证要求。
- 客户端使用代理卡片来发现代理并了解如何安全有效地与其交互。

🏠 想象你开了一家小店，门口挂个牌子写着：

- 店名：“智能旅行助手”
- 能干啥：订机票、做行程、查天气
- 怎么联系我：网址是 <https://travel-agent.example/.well-known/agent.json>
- 需要密码吗？需要（比如用 API Key）
- 支持边做边汇报进度吗？支持！

这个“牌子”就是 代理卡片——一个标准格式的 JSON 文件，别的 AI 一看到它，就知道：“哦，这家伙能帮我干啥、怎么调用、要不要认证”，不用瞎猜。

👉 作用：让别人轻松找到你、知道怎么跟你合作。



2. 任务：具体要完成的事情（生成旅游规划）

- 当客户端向代理发送消息时，代理可能会确定完成请求需要完成一个有状态的任务（例如，“生成报告”，“预订航班”，“回答问题”）。
- 每个任务都有代理定义的唯一 ID，并按照定义的生命周期（例如，已提交，进行中，需要输入，已完成，失败）进行进展。
- 任务是有状态的，可能涉及客户端和服务端之间的多个交换（消息）。

👤 用户说：“帮我规划一个去云南5天的旅行。”

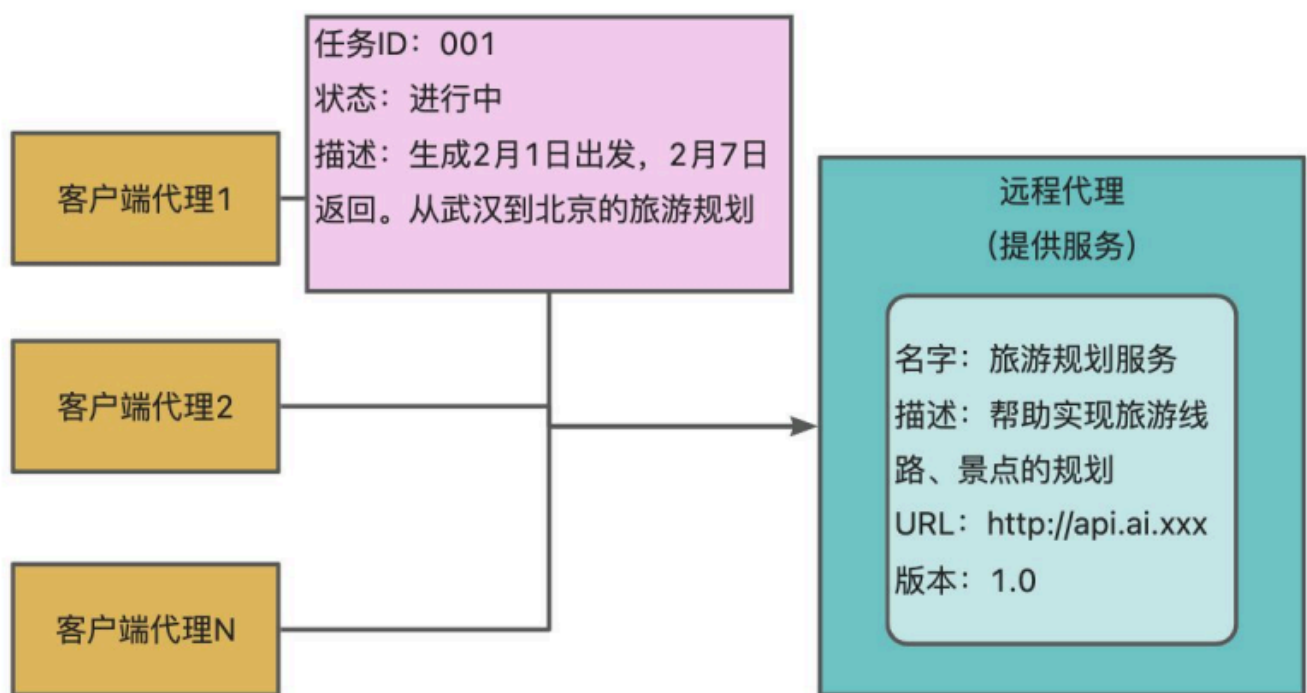
这可不是一句话就能回完的事，可能要查航班、订酒店、排日程……

于是客户端对远程代理说：“启动一个任务！”

代理就创建一个带唯一编号的任务（比如 `task-abc123`），状态从“已提交” → “进行中” → “需要你确认预算” → “已完成”。

整个过程可能来回沟通好几次，但都围绕同一个任务。

👉 作用：把复杂事情打包成一个“项目”，有始有终、可追踪、可中断、可继续。



3. 消息：代表客户端和代理之间的单个转向或通信单元。

- 消息具有 role（对于客户端发送的消息为 "user"，对于服务器发送的消息为 "agent"），并包含一个或多个携带实际内容的 Part 对象。
- Message 对象的 messageId 部分是由消息发送者设置的每个消息的唯一标识符。用于传达指令、上下文、问题、答案或状态更新，这些内容不一定是正式 Artifacts。



每次客户端和代理“说话”，都是一条消息。

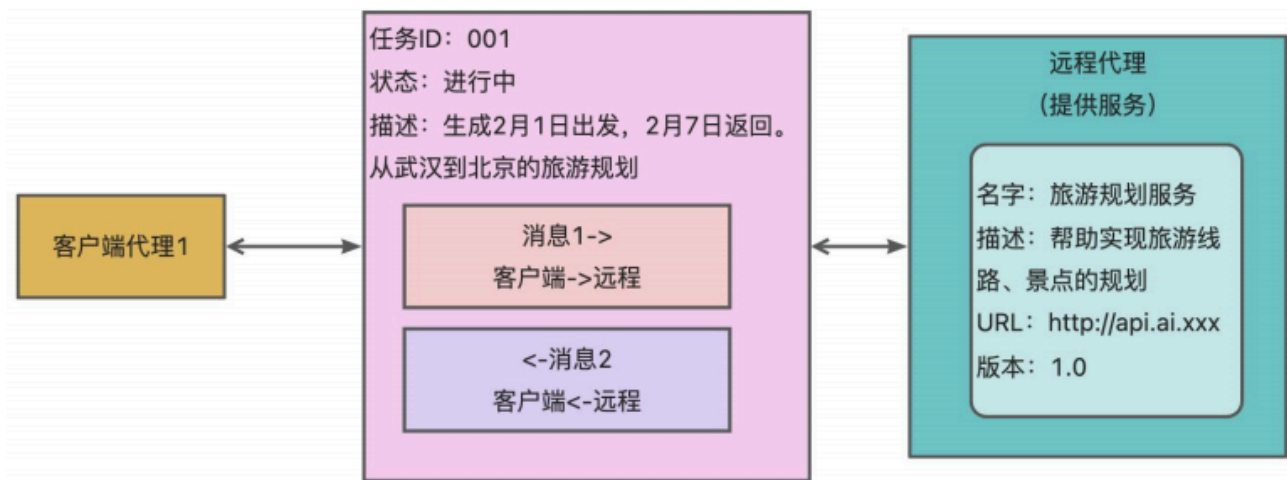
比如：

- 客户端发：“我想去大理，预算5000。”（角色是 `"user"`）
- 代理回：“好的，正在查机票……”（角色是 `"agent"`）

每条消息都有唯一 ID（比如 `msg-xyz789`），方便追踪。

消息里可以包含文字、文件、数据等（通过“部分”来装）。

👉 作用：对话的基本单位，像微信聊天里的一条消息。



4. 部分 (Part)：在消息或 Artifacts 中的基本内容单元。

- 每个部分具有特定的 type，可以携带不同类型的数据：
 - TextPart：包含纯文本内容。
 - FilePart：表示一个文件，可以作为内联 base64 编码的字节传输，也可以通过 URI 引用。包括文件名和媒体类型等元数据。
 - DataPart：携带结构化的 JSON 数据，适用于表单、参数或任何可机器读取的信息。

🗑 一条消息不能只塞一堆乱七八糟的东西，得分分类打包。

“部分”就是这些小包裹，每种类型干不同的事：

- TextPart：纯文字，比如“你好！”
- FilePart：文件，比如一张行程图（可以是 base64 编码直接传，也可以给个下载链接）
- DataPart：结构化数据，比如

```
{ "days": 5, "budget": 5000, "destinations": ["大理", "丽江"] }
```

你可以一条消息里同时发文字 + 文件 + 数据，就像微信里既能打字，又能发图片和位置。

👉 作用：让不同类型的内容各归其位，机器也能准确理解。

5. 工件 (Artifacts)：代表远程代理在处理任务过程中生成的有形输出或结果。

- 示例包括生成的文档、图像、电子表格、结构化数据结果，或任何其他作为任务直接结果的独立信息。
- Artifacts 由一个或多个 Part 对象组成，并且可以逐步流式传输。



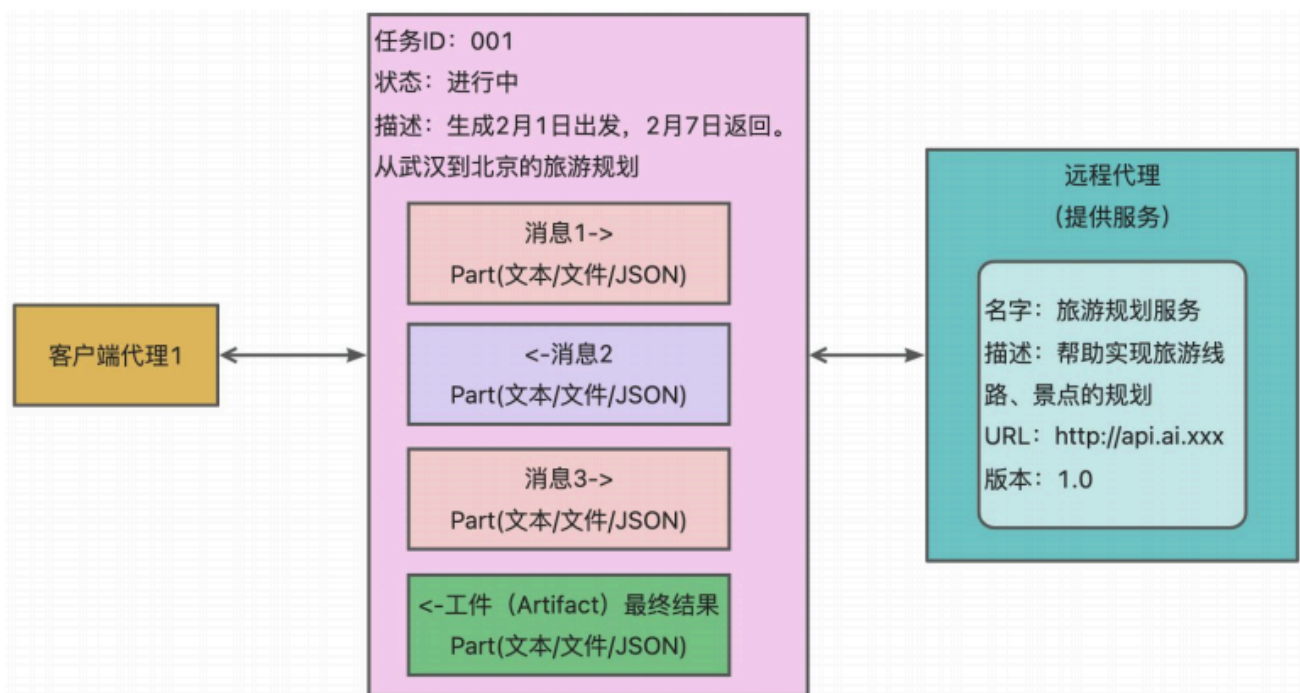
当任务快完成时，代理会生成一些实实在在的结果，比如：

- 一份 PDF 行程单
- 一张景点地图（图片）
- 一个 Excel 预算表
- 一段 JSON 格式的推荐列表

这些成果统称为 工件（Artifacts），它们由一个或多个“部分（Part）”组成。

而且可以边做边传——比如先给你文字摘要，再慢慢传高清图。

👉 作用：把任务的“交付物”标准化，方便接收方使用或展示。



总的来说

就像你找设计师做海报：

先看他的作品集（代理卡）→ 下单一个设计任务（任务）→ 中间聊需求、改稿（消息+部分）→ 最后他交给你 PNG 和源文件（工件）。

这就是 A2A 的基本通信逻辑——清晰、灵活、适合复杂协作！

2.4 A2A 协议核心概念之交互机制

客户端代理向服务端代理发送任务指令

服务端代理处理任务需要时间

服务端代理如何返回客户端代理结果？

1) 请求/响应（轮询）：

- 客户端发送请求（例如，使用 message/send RPC 方法），并从服务器接收响应。
- 执行的任务需要长时间运行，服务器代理以 working 状态进行响应。
- 客户端代理会定期调用 tasks/get 来轮询更新，直到任务达到终端状态（例如，completed，failed）。



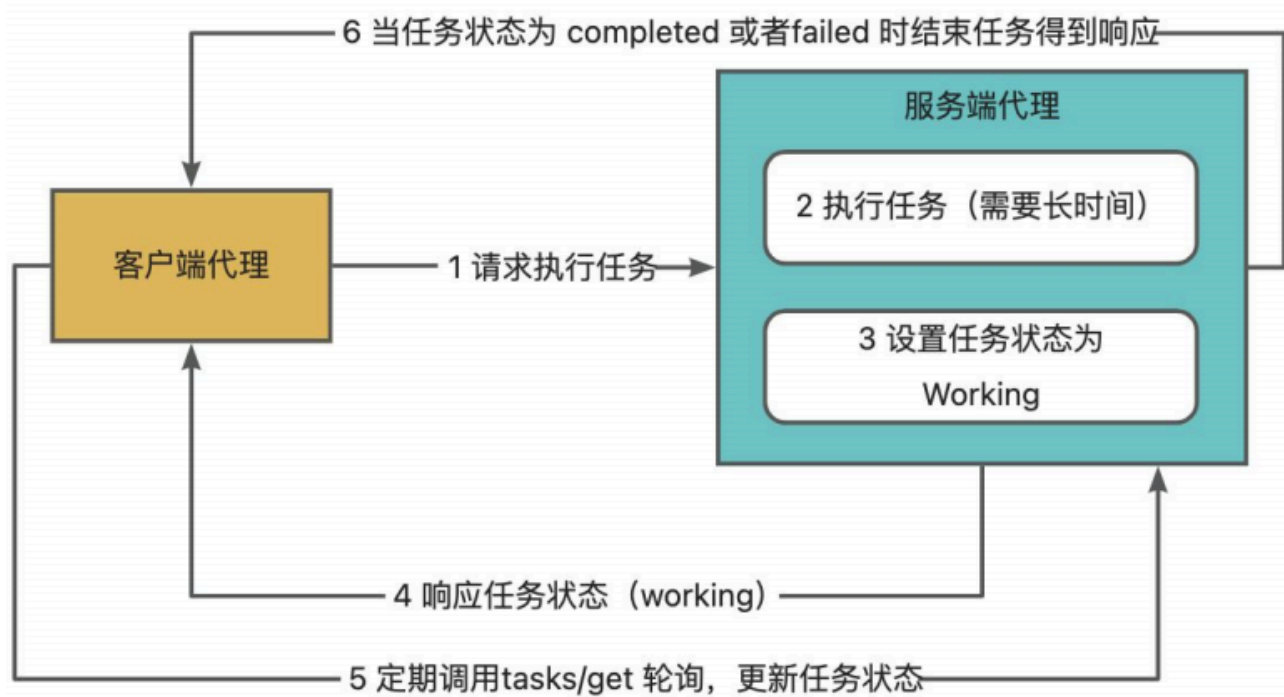
1 请求/响应（轮询）——像不断问“好了没？”

- 你发个请求：“开始做攻略！”
- 对方回你一句：“收到，正在处理中（working）。”
- 然后你就得自己每隔几秒主动去问一次：“好了吗？好了吗？”（调用 `tasks/get` 查状态）
- 直到对方说：“完成了！”或者“失败了”，你才算拿到结果。

✅ 优点：简单直接，兼容性好（所有系统都支持）。

❌ 缺点：你得一直盯着问，浪费网络和精力；还可能刚问完它就完成了，但你下一次才查到，有延迟。

👉 就像你在餐厅点了一份现烤披萨，每隔30秒就跑厨房门口问一次：“好了没？”



2) 流式传输（服务器发送事件 - SSE）：

- 服务端代理返回增量结果，或者提供实时进度更新的任务。
- 客户端代理使用 `message/stream` 与服务端代理发起交互。
- 服务端代理保持 HTTP 连接进行响应，发送服务器事件（SSE）。
- 事件可以是 任务、消息，或者 `TaskStatusUpdateEvent`（用于状态更改），或者 `TaskArtifactUpdateEvent`（用于新的或更新的工件块）。
- 服务器代理在代理卡中定义 `stream` 的能力。



2 流式传输（SSE）——像直播进度，边做边告诉你

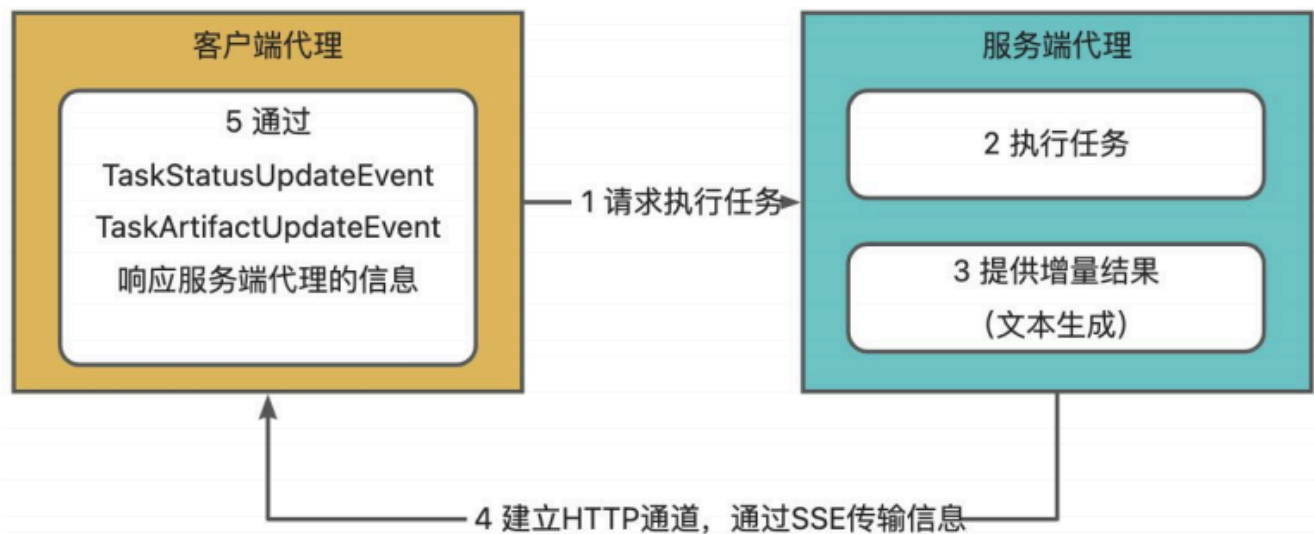
- 你发起请求时说：“我要用流式模式（`message/stream`）。”
- 服务端不挂电话，而是保持连接打开，像开直播一样：
 - 先发：“开始查航班……”
 - 再发：“酒店推荐已生成”
 - 接着发：“这是行程草稿（文字）”
 - 最后发：“攻略 PDF 已完成（文件）”

这些信息通过 SSE（Server-Sent Events）实时推送过来，你不用问，它主动说。

✅ 优点：实时性强，体验流畅，适合需要“边做边看”的场景。

❌ 缺点：需要保持长连接，网络不稳定时容易断；有些防火墙或代理不支持。

👉 就像厨师一边做菜一边用对讲机跟你同步：“现在煎牛排了” “酱汁调好了” “马上装盘！”



3) 推送通知:

- 对于非常长时间运行的任务或场景（任务跨越多个系统），或者维持持久连接（如 SSE）不切实际的情况。
- 客户在启动任务时（或通过调用 `tasks/pushNotificationConfig/set`）可以提供一个 Webhook URL。
- 当任务状态发生变化（例如完成、失败或需要输入）时，服务端代理调用客户端 Webhook 发送异步通知（HTTP POST 请求）。
- 服务器代理在代理卡中定义 `pushNotifications` 功能。



3 推送通知（Webhook）——像留个电话，好了打给你

- 你一开始就说：“我不等你，你做好了主动通知我。”
 - 你提供一个 回调网址（Webhook URL），比如 <https://your-app.com/task-done>
 - 然后你就去干别的事了。
 - 等服务端任务完成（或卡住了、需要你确认），它会主动发一个 HTTP 请求到你给的网址，说：“任务 abc123 已完成，请来取结果！”
- ✅ 优点：完全异步，不占连接，适合超长时间任务（比如几小时、几天），也适合跨系统协作。

❌ 缺点：你需要有个公网能访问的服务来接收通知；配置稍微复杂点。

👉 就像你把衣服送去干洗店，留了个电话，店员洗好后直接打电话：“您的衣服好了，来取吧！”——你不用一直蹲在店门口。

为什么需要支持三种？
因为现实中的 AI 代理协作场景千差万别：

场景	最佳交互方式
简单查询（如“今天天气？”）	请求/响应（一次搞定）
复杂生成（如写报告、做PPT）	流式传输（边生成边展示）
超长任务（如训练模型、人工审批流程）	推送通知（异步回调）
客户端网络不稳定或不能常在线	推送通知更可靠
前端网页应用想实时更新界面	流式传输体验最好

3. A2A 和 MCP 如何配合？

3.1 A2A 和 MCP 配合

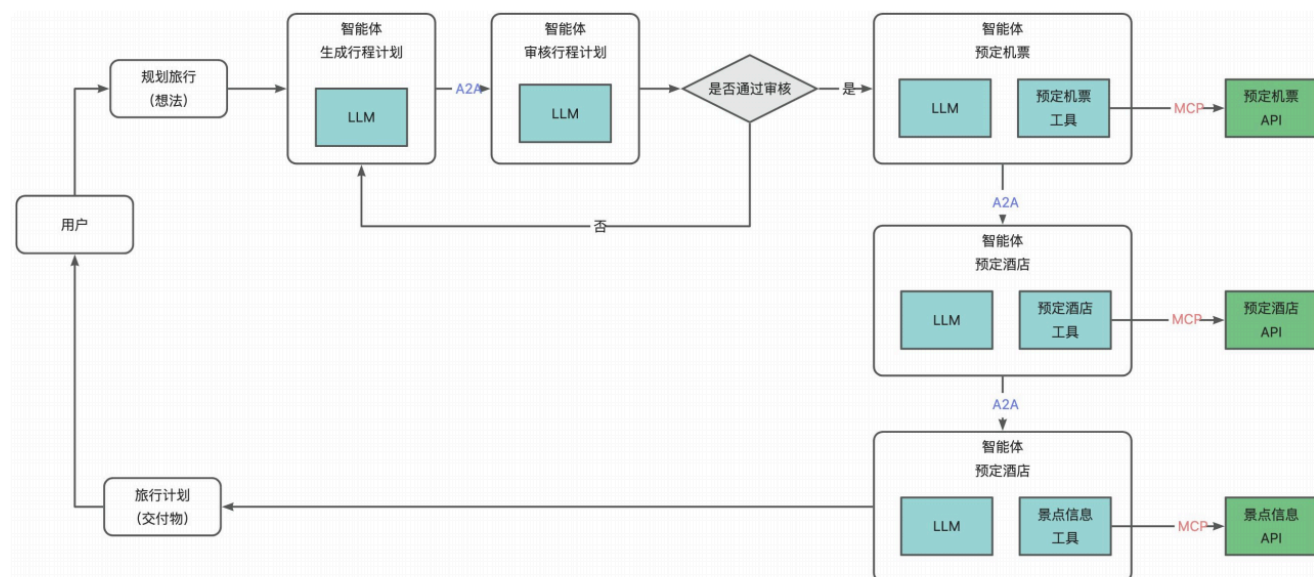
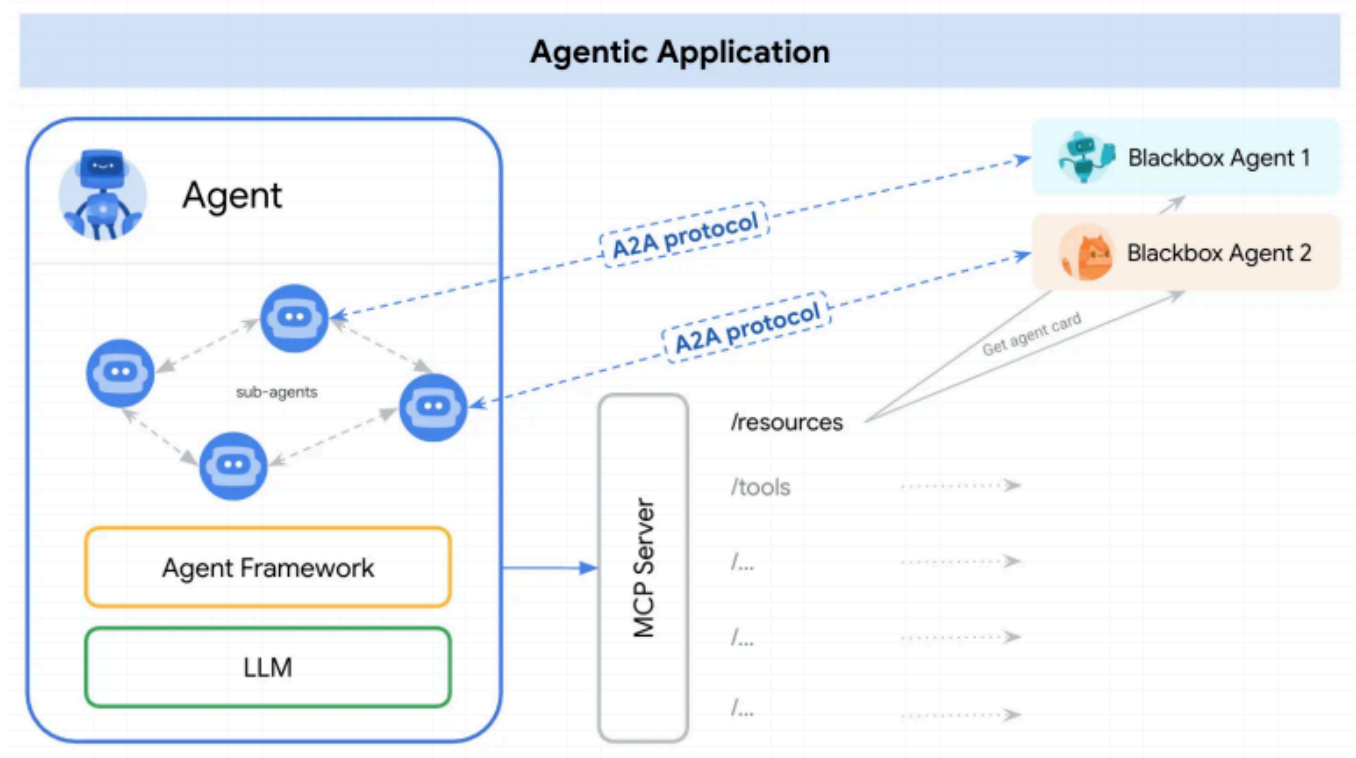
MCP（Model Context Proctocol）解决大模型对工具和资源的调用，MCP用于工具。

MCP 专注工具和资源：

1. 通过结构化的方式调用工具和资源，利用function call方式实现（例如，
2. 计算器、数据库查询 API、天气查找服务）。
3. 大模型通过对自然语言的判断，决定是否调用工具或者资源。
4. 调用工具或者资源的结果交给大模型进行润色返回用户。

A2A 专注代理协同：

1. 代理是自主的系统。结合推理、计划、工具、记忆等多种能力完成复杂工作，通常需要多轮对话。完成的任务复杂，任务时长不可控，采用技术不可控。
2. 需要多个代理参与，代理之间存在多轮交互。
3. 代理处理任务包含了工具和资源的调用



4. A2A 协议如何使用？

A2A官方提供的代理例子 <https://github.com/a2aproject/a2a-samples>

4.1 A2A 项目环境部署

通过conda 创建开发环境

代码块

```
1 conda create -n a2a python=3.11
```

激活 a2a 的环境

代码块

```
1 conda activate a2a
```

安装相关依赖

代码块

```
1 pip install -r samples/python/requirements.txt
```

安装的依赖内容如下：

代码块

```
1 a2a[http-server]>=0.3.0
2 uvicorn
```

a2a-sdk：这是 Google 官方推出的 Agent-to-Agent (A2A) Protocol

Python SDK，用于构建多智能体系统，实现模型与模型之间通过统一协议进行对话、上下文传递和协作。

uvicorn：是一款轻量级且极快的 ASGI Web 服务器，适用于运行异步 Python 应用（如 FastAPI、Starlette 等），支持 HTTP/1.1、WebSocket 等协议。

4.2 A2A 项目开发，部署和运行

agent_executor.py

代码块

```
1 # 导入 A2A 框架中执行代理任务所需的核心类
2 from a2a.server.agent_execution import AgentExecutor, RequestContext
3 # 导入用于发送事件（比如消息、进度更新）的队列工具
4 from a2a.server.events import EventQueue
5 # 导入一个便捷函数，用来创建一条“由代理发出的文本消息”
6 from a2a.utils import new_agent_text_message
7
```

```

8
9  # --8<-- [start:HelloWorldAgent]
10 # 定义一个最简单的 AI 代理：它啥也不干，只会说 "Hello World"
11 class HelloWorldAgent:
12     """Hello World Agent."""
13
14     # invoke 是这个代理的核心方法，调用它就会返回一句问候语
15     async def invoke(self) -> str:
16         return 'Hello World'
17 # --8<-- [end:HelloWorldAgent]
18
19
20 # --8<-- [start:HelloWorldAgentExecutor_init]
21 # 这个类是 HelloWorldAgent 的“执行器”——负责在 A2A 系统里运行它
22 # 它继承自 A2A 提供的标准基类 AgentExecutor
23 class HelloWorldAgentExecutor(AgentExecutor):
24     """Test AgentProxy Implementation."""
25
26     # 初始化时，创建一个 HelloWorldAgent 实例，存起来备用
27     def __init__(self):
28         self.agent = HelloWorldAgent()
29     # --8<-- [end:HelloWorldAgentExecutor_init]
30
31     # --8<-- [start:HelloWorldAgentExecutor_execute]
32     # execute 是 A2A 框架调用的核心方法：当有任务进来，就执行这个函数
33     async def execute(
34         self,
35         context: RequestContext,      # 包含任务上下文信息（比如用户ID、任务ID等）
36         event_queue: EventQueue,      # 一个“消息通道”，用来把结果或进度发出去
37     ) -> None:
38         # 调用我们自己的代理，让它干活（这里就是返回 "Hello World"）
39         result = await self.agent.invoke()
40         # 把结果包装成一条“代理发出的文本消息”，放进事件队列，通知客户端
41         # 内容都不是直接 return 给调用者，而是通过 event_queue.enqueue_event(...)
“扔进队列”，
42         # 由 A2A 框架后台统一处理并按协议规则（轮询/SSE/Webhook）发送给客户端。
43         await event_queue.enqueue_event(new_agent_text_message(result))
44     # --8<-- [end:HelloWorldAgentExecutor_execute]
45
46     # --8<-- [start:HelloWorldAgentExecutor_cancel]
47     # 如果用户中途想取消任务，框架会调用 cancel 方法
48     async def cancel(
49         self, context: RequestContext, event_queue: EventQueue
50     ) -> None:
51         # 但这个简单代理不支持取消，所以直接抛个异常（实际项目中可优化）
52         raise Exception('cancel not supported')

```

☕ 场景：设计一个只会说 “Hello World!” 的 AI 咖啡师

👤 1. HelloWorldAgent → 真正的 “AI 咖啡师本人”

- 这个咖啡师非常特别：他不会做咖啡、不会拉花、甚至不会倒水。
- 但只要你问他：“你好吗？”、“能帮我查汇率吗？”、“给我一杯拿铁？”，他永远只回答一句：“Hello World!”
- 他的核心能力就藏在 `invoke()` 方法里——这是他唯一的“技能按钮”。

💡 就像一个机器人，出厂设定就是复读机，但复读的内容是程序员定的。

🔧 2. HelloWorldAgentExecutor → “AI 咖啡师的操作台 + 对讲机系统”

这个执行器不是咖啡师本人，而是连接咖啡师和整个店铺运营系统的桥梁。它有三个关键功能：

✅ 初始化（`init`）→ 给操作台装上咖啡师

- 就像开店前，你把那个只会说 “Hello World!” 的 AI 咖啡师固定安装在后厨的操作台上。
- 以后所有订单来了，都通过这个操作台喊他干活。

🔧 执行任务（`execute`）→ 接单 → 喊话 → 发结果

- 当店长（`DefaultRequestHandler`）送来一张订单（比如“用户问：10美元兑多少卢比？”），操作台立刻：
 - a. 按下咖啡师的“技能按钮”（调用 `agent.invoke()`）；
 - b. 咖啡师大喊：“Hello World!”；
 - c. 操作台不直接把话传给顾客，而是拿起对讲机，把这句话发到中央广播系统（`event_queue`）；
 - d. 店里的广播系统（A2A 框架）会自动决定：是打包成一次性消息，还是分段直播给顾客。

💡 重点：结果不是 `return` 出去的，而是“扔进事件队列”——这是为了支持流式、异步、多客户端通知。

🚫 取消任务（`cancel`）→ 试图打断咖啡师说话

- 顾客突然说：“算了，别说了！”
- 但你的 AI 咖啡师是个“一根筋”——一旦开口，必须说完 “Hello World!” 才停。
- 所以操作台只能无奈回复：“对不起，这位咖啡师不支持中途打断！”

💡 实际项目中，复杂任务（如生成长文）会在这里清理资源、终止循环，但 “Hello World” 太简单，没必要。

✈️ 整个协作流程（从顾客下单到收到回复）

1. 顾客点单 → 店长接单 → 创建任务 → 放入小票夹（`InMemoryTaskStore`）
2. 店长通知后厨：“有新单！请 `HelloWorldAgentExecutor` 处理！”
3. 执行器（操作台）启动：
 - 唤醒 `HelloWorldAgent`（AI 咖啡师）
 - 获取他说的 “Hello World!”
 - 把这句话塞进 事件队列（对讲机广播系统）
4. A2A 框架监听队列：
 - 如果是普通请求 → 等全部说完，打包回复；
 - 如果是流式请求 → 每个字/词都实时推送（虽然这里只有一句）；
5. 顾客收到结果：“Hello World!”

✓ 为什么这样设计？

- 解耦：咖啡师（业务逻辑）不用知道 “怎么对外说话”，只管输出内容；
- 标准化：所有代理都通过 `event_queue` 发消息，框架统一处理协议细节；
- 可扩展：未来换成 “会算汇率的咖啡师”，只需换 `agent`，执行器结构不变。

就像麦当劳的后厨：

不管做汉堡还是冰淇淋，都有标准操作台、标准出餐口、标准叫号系统——`AgentExecutor` 就是这个标准化的 “AI 后厨工作站”。

__main__

代码块

```
1  # 导入 uvicorn, 这是一个用于运行异步 Python Web 应用的轻量级服务器
2  import uvicorn
3  # 从 A2A 框架中导入用于构建符合 A2A 协议的 Starlette 应用的核心类
4  from a2a.server.apps import A2AStarletteApplication
5  # 导入默认的请求处理器，它负责解析客户端发来的 A2A 请求并调用对应的代理执行器
6  from a2a.server.request_handlers import DefaultRequestHandler
7  # 导入一个基于内存的任务存储实现，用于临时保存任务的状态（适合开发和测试）
8  from a2a.server.tasks import InMemoryTaskStore
9  # 从类型模块导入描述代理能力所需的数据结构：代理能力、代理名片、代理技能
10 from a2a.types import (AgentCapabilities, AgentCard, AgentSkill,)
11 # 导入我们自己实现的代理执行器（即前面定义的 HelloWorldAgentExecutor），type: ignore
   是为了忽略类型检查警告
12 from agent_executor import (HelloWorldAgentExecutor, # type: ignore[import-
   untyped])
13
14 # 当这个文件被直接运行时（而不是被其他文件导入），才执行下面的主程序逻辑
```

```

15 if __name__ == '__main__':
16     # 定义一个基础技能: ID 为 'hello_world', 表示这个代理能做“返回 Hello World”这件事
17     skill = AgentSkill(
18         id='hello_world', # 技能的唯一标识符, 其他代理通过这个 ID 调用该技能
19         name='Returns hello world', # 技能的可读名称
20         description='just returns hello world', # 对技能功能的详细说明
21         tags=['hello world'], # 给技能打标签, 便于发现和分类
22         examples=['hi', 'hello world'], # 提供调用该技能的示例输入, 帮助用户理解怎
        么用
23     )
24     # 定义一个扩展技能: 只有经过身份验证的用户才能使用的“超级打招呼”功能
25     extended_skill = AgentSkill(
26         id='super_hello_world',
27         name='Returns a SUPER Hello World',
28         description='A more enthusiastic greeting, only for authenticated
        users.',
29         tags=['hello world', 'super', 'extended'],
30         examples=['super hi', 'give me a super hello'],
31     )
32     # 创建公开版的代理名片 (所有人都能看到的版本)
33     public_agent_card = AgentCard(
34         name='Hello World Agent', # 代理的名称
35         description='Just a hello world agent', # 代理的简要描述
36         url='http://localhost:9999/', # 代理服务的根 URL, 客户端通过这个地址与之通
        信
37         version='1.0.0', # 代理的版本号
38         default_input_modes=['text'], # 默认接受的输入类型是文本
39         default_output_modes=['text'], # 默认输出的类型也是文本
40         capabilities=AgentCapabilities(streaming=True), # 声明该代理支持流式传输
        (如 SSE)
41         skills=[skill], # 公开版只暴露基础技能, 隐藏高级功能
42         supports_authenticated_extended_card=True, # 表示: 如果用户已认证, 还可以
        提供一张功能更全的名片
43     )
44     # 基于公开名片复制出一张“认证用户专用”的增强版名片, 并更新部分内容
45     specific_extended_agent_card = public_agent_card.model_copy(
46         update={
47             'name': 'Hello World Agent - Extended Edition', # 改个名字以示区别
48             'description': 'The full-featured hello world agent for
        authenticated users.', # 更详细的描述
49             'version': '1.0.1', # 可以使用不同的版本号
50             # 其他字段 (如 url、capabilities、default_input_modes 等) 自动继承自
        public_agent_card
51             'skills': [skill, extended_skill], # 同时包含基础技能和扩展技能, 功能
        更强大
52         }
53     )

```

```

54     # 创建请求处理器实例，它会把收到的 A2A 请求交给 HelloWorldAgentExecutor 处理，并
    用内存存储任务状态
55     request_handler = DefaultRequestHandler(
56         agent_executor=HelloWorldAgentExecutor(), # 使用我们自定义的代理执行器来运
    行任务
57         task_store=InMemoryTaskStore(), # 任务数据暂存在内存中（重启服务后会丢失，
    仅用于测试）
58     )
59     # 构建完整的 A2A Web 应用：绑定公开名片、请求处理器和认证用户的增强名片
60     server = A2AStarletteApplication(
61         agent_card=public_agent_card, # 所有人都能看到的代理信息
62         http_handler=request_handler, # 处理 HTTP 请求的实际逻辑
63         extended_agent_card=specific_extended_agent_card, # 已认证用户可访问的完
    整功能名片
64     )
65     # 启动 Web 服务：将构建好的应用跑在本机 9999 端口，允许任何 IP 地址访问（0.0.0.0
    表示监听所有网络接口）
66     uvicorn.run(server.build(), host='0.0.0.0', port=9999)

```

代码块

```

1  A2AStarletteApplication
2  |
3  |—— 对外提供 /agent.json（代理名片）
4  |—— 注册所有 A2A 标准 API 路由（/tasks, /message/...）
5  |—— 所有业务请求 → 转交给 → DefaultRequestHandler
6  |
7  |—— 调用 HelloWorldAgentExecutor.execute()
8  |—— 使用 InMemoryTaskStore 存任务状态

```



定义了两个代理技能（干什么）：

基础技能：返回简单的"hello world"问候-普通

扩展技能：返回更热情的问候-特殊

创建了两个代理卡片（描述服务）：

公共卡片：包含基础技能，面向所有用户

扩展卡片：包含基础技能和扩展技能，面向认证用户

使用A2A框架构建了一个Web服务：

使用DefaultRequestHandler处理请求

使用InMemoryTaskStore存储任务

使用HelloWorldAgentExecutor执行代理逻辑

通过A2AStarletteApplication包装成Starlette应用

最后使用uvicorn运行服务：

监听localhost:9999端口

☕ 场景：你开了一家“智能咖啡店”（这就是你的 Web 服务）

1. HelloWorldAgentExecutor → 咖啡师本人

- 他是真正会做咖啡的人。
- 你问他：“能做杯拿铁吗？”
他只会回答一句：“Hello World!”（在这个例子里，他其实不会做咖啡，只会打招呼 😊）。
- 但他不直接面对顾客，只听从店长安排。

2. InMemoryTaskStore → 店里的点单小票夹（挂在墙上那种）

- 每当有顾客下单，店长就把订单写在小票上，夹在这个夹子上。
- 咖啡师做完一杯，就从小票夹里取下对应的单子。
- 注意：这个夹子是纸质的、临时的——如果店里停电或关门重启，所有小票就丢了（因为是“内存存储”，不是数据库）。

3. DefaultRequestHandler → 店长（负责调度）

- 顾客进门点单，店长接待。
- 他干三件事：
 - 把订单写到小票上，夹进 InMemoryTaskStore；
 - 喊一声：“咖啡师，有新单！”（调用 HelloWorldAgentExecutor.execute()）；
 - 等咖啡师做好后，把结果（“Hello World!”）端给顾客。
- 他不管店面装修、招牌怎么挂、要不要会员卡，只管“接单 → 安排做 → 送结果”。

4. A2AStarletteApplication → 整个咖啡店的营业执照 + 门头 + 菜单牌 + 收银系统

- 它决定了：
 - 店名叫什么（name='Hello World Agent'）；
 - 菜单上写“本店支持现场观看制作过程”（streaming=True）；
 - 挂出公开菜单（agent_card），还有一份“会员专享菜单”（extended_agent_card）；
 - 所有顾客都通过同一个入口（HTTP 接口）进来；
 - 把真正的接待工作交给店长（http_handler=request_handler）。
- 它是对外的完整形象，让路人知道：“这是一家合规、功能明确的智能咖啡店”。

📄 客户端来点单时发生了什么？

1. 顾客（客户端）走到店门口，先看菜单（访问 `/well-known/agent.json`）；
2. 决定点一杯“打招呼咖啡”（发送 `message/send` 请求）；
3. 咖啡店（`A2AStarletteApplication`）接待顾客，引导到店长那里；
4. 店长（`DefaultRequestHandler`）：
 - 写张小票（存入 `InMemoryTaskStore`）；
 - 叫咖啡师开工；
5. 咖啡师（`HelloWorldAgentExecutor`）：
 - 大喊一声：“Hello World!”；
 - 把这句话交给店长；
6. 店长把这句话装进杯子，递给顾客——任务完成！

如果顾客说：“我要边看边喝！”（流式请求），

咖啡师就会分几次喊：“Hel… lo… World…!”，店长也分几次递纸条给顾客。

✅ 总结对应关系：

A2A 组件	咖啡店角色	职责
HelloWorldAgentExecutor	咖啡师	执行核心业务逻辑
InMemoryTaskStore	点单小票夹	临时记录任务状态
DefaultRequestHandler	店长	接单、派单、交货
A2AStarletteApplication	整个咖啡店	对外提供标准化服务

这样设计的好处是：

- 想换咖啡师？没问题，只要他会说“Hello World”就行；
- 想把小票夹换成电子屏（换成数据库）？店长不用改；
- 想开连锁店？复制整个“咖啡店”模板即可！

这就是 A2A 框架的模块化、标准化、可扩展之美。

test_client.py

代码块

```
1  # 导入日志模块，用来打印运行过程中的信息（比如“正在连接...”、“收到结果”等）
2  import logging
3
4  # 从 typing 模块导入 Any，表示“任意类型”，用于类型提示
```

```

5  from typing import Any
6  # 从 uuid 模块导入 uuid4, 用于生成唯一的随机 ID (比如消息ID、请求ID)
7  from uuid import uuid4
8
9  # 导入 httpx, 这是一个支持异步 HTTP 请求的客户端库 (类似 requests, 但支持
    async/await)
10 import httpx
11
12 # 从 A2A 客户端模块导入两个核心类:
13 # - A2ACardResolver: 用于自动获取远程代理的“名片” (Agent Card)
14 # - A2AClient: 用于向远程代理发送任务请求
15 from a2a.client import A2ACardResolver, A2AClient
16 # 从类型定义中导入客户端需要的数据结构
17 from a2a.types import (
18     AgentCard,                # 代理名片的数据结构
19     MessageSendParams,        # 发送消息时的参数格式
20     SendMessageRequest,        # 一次普通 (非流式) 消息请求的封装
21     SendStreamingMessageRequest, # 一次流式消息请求的封装
22 )
23 # 从常量模块导入 A2A 协议规定的标准路径
24 from a2a.utils.constants import (
25     AGENT_CARD_WELL_KNOWN_PATH,    # 公开代理名片的标准路径: /.well-
    known/agent.json
26     EXTENDED_AGENT_CARD_PATH,      # 认证用户专用名片的路径: /agent-card-extended
27 )
28
29
30 # 定义主异步函数 main(), 整个客户端逻辑都在这里
31 async def main() -> None:
32     # 设置日志级别为 INFO, 这样 info/warning/error 级别的日志都会显示出来
33     logging.basicConfig(level=logging.INFO)
34     # 获取一个专属的日志记录器, 名字是当前模块 (__name__)
35     logger = logging.getLogger(__name__)
36
37     # --8<-- [start:A2ACardResolver]
38     # 设置要连接的代理服务地址 (就是之前启动的 HelloWorld 服务)
39     base_url = 'http://localhost:9999'
40
41     # 创建一个异步 HTTP 客户端 (httpx.AsyncClient), 用于发起网络请求
42     async with httpx.AsyncClient() as httpx_client:
43         # 初始化 A2A 名片解析器: 它知道怎么从指定地址获取代理的公开或扩展名片
44         resolver = A2ACardResolver(
45             httpx_client=httpx_client, # 传入 HTTP 客户端, 用于发请求
46             base_url=base_url,         # 代理服务的基础 URL
47             # agent_card_path 和 extended_agent_card_path 使用默认值 (即标准路径)
48         )
49     # --8<-- [end:A2ACardResolver]

```

```

50
51     # 声明一个变量，用来保存最终决定使用的代理名片（可能是公开版，也可能是扩展版）
52     final_agent_card_to_use: AgentCard | None = None
53
54     try:
55         # 打印日志：准备获取公开名片
56         logger.info(
57             f'Attempting to fetch public agent card from: {base_url}
{AGENT_CARD_WELL_KNOWN_PATH}'
58         )
59         # 调用 resolver 获取公开名片（默认从 /.well-known/agent.json 读取）
60         _public_card = await resolver.get_agent_card()
61         # 成功获取后，打印名片内容（格式化成 JSON，方便阅读）
62         logger.info('Successfully fetched public agent card:')
63         logger.info(_public_card.model_dump_json(indent=2,
exclude_none=True))
64         # 默认先用公开名片
65         final_agent_card_to_use = _public_card
66         logger.info('\nUsing PUBLIC agent card for client initialization
(default).')
67
68         # 检查公开名片是否声明“支持认证用户的扩展名片”
69         if _public_card.supports_authenticated_extended_card:
70             try:
71                 # 如果支持，就尝试带上认证令牌去获取扩展名片
72                 logger.info(
73                     f'\nPublic card supports authenticated extended card.
Attempting to fetch from: {base_url}{EXTENDED_AGENT_CARD_PATH}'
74                 )
75                 # 构造带认证头的 HTTP 请求头（这里用 dummy-token 模拟真实
token)
76                 auth_headers_dict = {
77                     'Authorization': 'Bearer dummy-token-for-extended-card'
78                 }
79                 # 调用 get_agent_card 并指定路径和认证头，获取扩展名片
80                 _extended_card = await resolver.get_agent_card(
81                     relative_card_path=EXTENDED_AGENT_CARD_PATH, # 指定路
径 /agent-card-extended
82                     http_kwargs={'headers': auth_headers_dict}, # 附带认证
头
83                 )
84                 # 成功获取扩展名片，打印内容
85                 logger.info('Successfully fetched authenticated extended
agent card:')
86                 logger.info(_extended_card.model_dump_json(indent=2,
exclude_none=True))
87                 # 更新最终使用的名片为扩展版

```

```

88         final_agent_card_to_use = _extended_card
89         logger.info('\nUsing AUTHENTICATED EXTENDED agent card for
client initialization.')
90     except Exception as e_extended:
91         # 如果获取扩展名片失败（比如没权限、服务没实现），就降级回公开名片
92         logger.warning(
93             f'Failed to fetch extended agent card: {e_extended}.
Will proceed with public card.',
94             exc_info=True,
95         )
96     elif _public_card:
97         # 如果公开名片明确说“不支持扩展名片”，就安心用公开版
98         logger.info('\nPublic card does not indicate support for an
extended card. Using public card.')
99
100     except Exception as e:
101         # 如果连公开名片都拿不到，说明服务根本不可用，直接报错退出
102         logger.error(f'Critical error fetching public agent card: {e}',
exc_info=True)
103         raise RuntimeError('Failed to fetch the public agent card. Cannot
continue.') from e
104
105     # --8<-- [start:send_message]
106     # 用最终确定的代理名片初始化 A2A 客户端
107     client = A2AClient(
108         httpx_client=httpx_client,          # 复用之前的 HTTP 客户端
109         agent_card=final_agent_card_to_use  # 告诉客户端这个代理能干啥、怎么调
用
110     )
111     logger.info('A2AClient initialized.')
112
113     # 构造一条用户消息的内容：角色是 user，内容是一段文本“10美元兑多少卢比？”
114     send_message_payload: dict[str, Any] = {
115         'message': {
116             'role': 'user', # 表示这是用户发的消息
117             'parts': [      # 消息可以包含多个“部分”，这里只有一个文本部分
118                 {'kind': 'text', 'text': 'how much is 10 USD in INR?'}
119             ],
120             'messageId': uuid4().hex, # 生成唯一消息ID，用于追踪
121         },
122     }
123     # 把 payload 包装成 A2A 协议要求的 SendMessageRequest 格式（带请求ID）
124     request = SendMessageRequest(
125         id=str(uuid4()),          # 整个请求的唯一ID
126         params=MessageSendParams(**send_message_payload) # 参数部分
127     )
128

```

```

129         # 发送普通（非流式）请求，并等待完整响应
130         response = await client.send_message(request)
131         # 打印返回结果（转成 JSON 格式，去掉空字段）
132         print(response.model_dump(mode='json', exclude_none=True))
133         # --8<-- [end:send_message]
134
135         # --8<-- [start:send_message_streaming]
136         # 再构造一个**流式请求**（同样的内容，但希望边做边返回）
137         streaming_request = SendStreamingMessageRequest(
138             id=str(uuid4()), # 新的请求ID
139             params=MessageSendParams(**send_message_payload) # 同样的消息内容
140         )
141
142         # 调用流式发送方法，它返回一个异步迭代器（不是立即结果！）
143         stream_response = client.send_message_streaming(streaming_request)
144
145         # 用 async for 循环逐块接收服务器推送的事件（比如进度、中间结果、最终答案）
146         async for chunk in stream_response:
147             # 每收到一块数据，就打印出来（也是 JSON 格式）
148             print(chunk.model_dump(mode='json', exclude_none=True))
149         # --8<-- [end:send_message_streaming]
150
151
152     # 当这个文件被直接运行时，启动异步主函数
153     if __name__ == '__main__':
154         # 导入 asyncio，用于运行异步函数
155         import asyncio
156         # 启动 main() 函数（它会自动处理所有 await 操作）
157         asyncio.run(main())

```

你是一位聪明的顾客（客户端），走进一家新开的“智能咖啡店”（A2A 代理服务），想点一杯咖啡，并体验它的全部功能。

 **场景：你作为顾客，去一家“智能咖啡店”点单**

第一步：先看门口的菜单牌（获取代理名片）

- 你走到店门口，看到一块标准招牌写着：

“本店菜单请查看：/.well-known/agent.json”（这是 A2A 协议规定的公开名片路径）。
- 你拿出手机（相当于 `A2ACardResolver`），访问这个地址，下载了一份公开菜单：
 - 店名：Hello World 咖啡
 - 能力：会说 “Hello World!”
 - 支持边做边看（流式输出）
 - 特别注明：“会员可查看隐藏菜单”

💡 这对应代码中的：

```
resolver.get_agent_card() → 获取 _public_card
```

第二步：尝试刷会员卡看隐藏菜单（获取扩展名片）

- 你正好有张会员卡（模拟 `Authorization: Bearer dummy-token...`），于是问店员：
 - “我能看看会员专属菜单吗？”
- 店员带你到后门，给你看了 扩展菜单：
 - 新增技能：“SUPER Hello World!!!”
 - 描述：“仅限认证用户，热情加倍！”
- 你决定：就用这份高级菜单点单！

💡 对应代码：

带 `headers` 调用

```
resolver.get_agent_card(relative_card_path=EXTENDED_AGENT_CARD_PATH)
```

成功后，`final_agent_card_to_use = _extended_card`

❗ 如果没会员卡或店员说“没有隐藏菜单”，你就安心用公开菜单——这就是代码里的降级逻辑。

第三步：正式点单（初始化 A2A 客户端）

- 你拿着最终确定的菜单（`final_agent_card_to_use`），走到收银台。
- 收银系统（`A2AClient`）根据菜单知道：
 - 这家店接受文本输入；
 - 支持普通下单和“边做边看”模式；
 - 请求要发到 <http://localhost:9999/message/send> 等接口。

💡 对应：`client = A2AClient(httpx_client, agent_card=...)`

第四步：第一次点单 —— 普通模式（非流式）

- 你说：“我想知道 10 美元能换多少印度卢比？”（虽然这跟咖啡无关，但这家店只会回答“Hello World!” 😊）
- 收银系统生成一张带唯一编号的订单（`request.id = uuid4()`），并记录你的问题（`messageId` 也唯一）。
- 你安静等待……几秒后，店员端来一个杯子，里面只有一张纸条写着：

📄 代码块 `"result": "Hello World!" }`

- 你拍照存档（`print(response...)`）。

💡 对应：`await client.send_message(request)`

第五步：第二次点单 —— “边做边看” 模式（流式）

- 你觉得有趣，又说：“这次我要看着你做！”
- 店员点头，开始“表演”：
 - 先举起牌子：“Hel…”
 - 再举：“lo…”
 - 然后：“World!”
 - 最后：🎉 Done!”
- 你每看到一块牌子，就记下来（`async for chunk in stream_response` → `print(chunk...)`）。

💡 这就是流式响应：服务器分多次推送事件，客户端逐块接收，适合长任务或实时反馈。

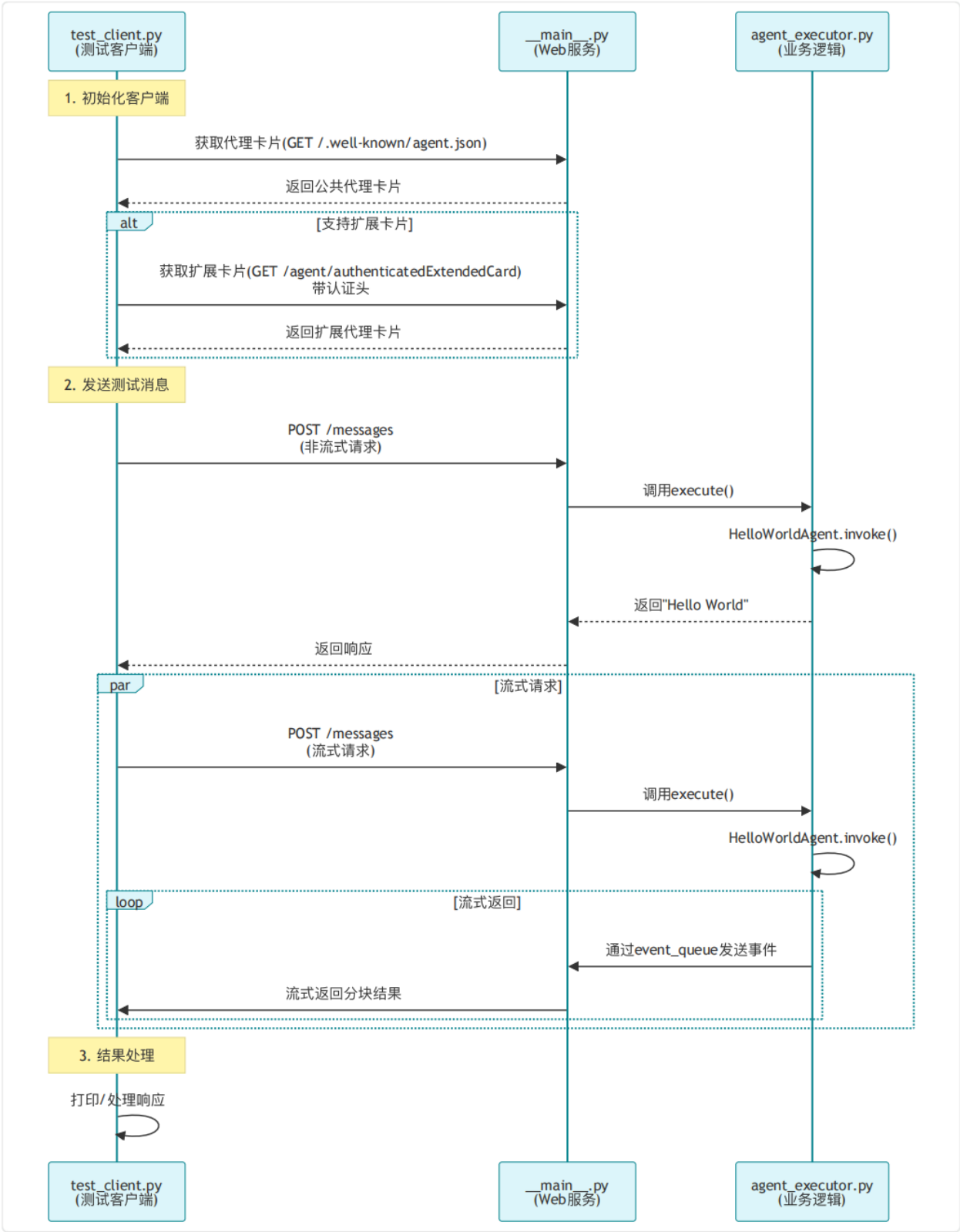
📄 整个流程总结（顾客视角 vs 代码）

你的行为（生活）	对应代码逻辑
走到店门口	<code>base_url = 'http://localhost:9999'</code>
查看公开菜单	<code>resolver.get_agent_card()</code>
刷会员卡看隐藏菜单	带 Authorization 头请求 <code>/agent-card-extended</code>
决定用哪份菜单	<code>final_agent_card_to_use = ...</code>
拿着菜单去点单	<code>A2AClient(...)</code>
说“帮我查汇率”	构造 <code>send_message_payload</code>
等待完整结果	<code>await client.send_message(...)</code>
要求“边做边看”	<code>client.send_message_streaming(...)</code>
一块一块记下过程	<code>async for chunk in stream_response: print(...)</code>

✅ 为什么这样设计？

- 标准化：所有智能代理都按同一套“菜单格式”（Agent Card）对外，顾客（客户端）不用重新学习每家店。
- 灵活升级：公开菜单 + 会员菜单，兼顾开放性和权限控制。
- 两种体验：普通模式适合简单问答，流式模式适合复杂任务（如写文章、跑代码）。

就像你去星巴克，不管哪家门店，都知道怎么点单、怎么用会员、怎么选堂饮外带——A2A 就是为 AI 代理世界建立的“连锁咖啡店标准” ☕🤖



4.3 基于 LangGraph 的A2A 案例实践

需求：

往往在 A2A 中需要通过完成任务来响应客户端代理的请求，但是一次对话往往无法完成任务。当任务需要更多消息的时候，就需要更多的用户输入信息。

例如：“美元的汇率是多少？”，问题中没有制定美元对什么货币的汇率，就需要通过多轮对话的方式再次询问用户。

此时，就需要服务端代理利用事件队列和任务状态的方式来处理多轮对话，通过判断任务执行情况来更新任务状态（input-required, completed）

任务执行情况和状态的变更通过事件队列完成同步。

agent.py

代码块

```
1  # 导入操作系统模块，用于读取环境变量（如 API 密钥、模型地址等）
2  import os
3
4  # 导入异步可迭代类型，用于支持流式响应（像聊天一样逐步返回结果）
5  from collections.abc import AsyncIterable
6
7  # 导入通用类型和字面量类型，用于类型提示
8  from typing import Any, Literal
9
10 # 导入 httpx 库，用于发送 HTTP 请求（比如调用汇率 API）
11 import httpx
12
13 # 从 LangChain 导入消息类型：AI 助手的消息和工具调用返回的消息
14 from langchain_core.messages import AIMessage, ToolMessage
15
16 # 从 LangChain 导入 tool 装饰器，用于将普通函数注册为 AI 可调用的工具
17 from langchain_core.tools import tool
18
19 # （虽然导入但未使用）用于调用 Google 的 Gemini 模型
20 from langchain_google_genai import ChatGoogleGenerativeAI
21
22 # 用于调用 OpenAI 兼容的大语言模型（包括本地部署的模型）
23 from langchain_openai import ChatOpenAI
24
25 # 导入内存检查点保存器，用于记住不同用户的对话历史
26 from langgraph.checkpoint.memory import MemorySaver
27
28 # 快速创建一个能“思考+调用工具”的智能体（ReAct 模式）
29 from langgraph.prebuilt import create_react_agent
30
31 # 用于定义结构化数据模型（比如规定响应格式）
32 from pydantic import BaseModel
```

```

33
34
35 # 创建一个内存检查点实例，用来保存每个用户的对话状态（通过 context_id 区分）
36 memory = MemorySaver()
37
38
39 # 使用 @tool 装饰器将函数注册为 AI 可调用的工具
40 @tool
41 def get_exchange_rate(
42     currency_from: str = 'USD',      # 默认从美元开始换
43     currency_to: str = 'EUR',        # 默认换成欧元
44     currency_date: str = 'latest',   # 默认查最新汇率
45 ):
46     """Use this to get current exchange rate.
47
48     Args:
49         currency_from: 要兑换的源货币（如 "USD"）
50         currency_to: 目标货币（如 "INR"）
51         currency_date: 查询日期，"latest" 表示最新汇率
52
53     Returns:
54         包含汇率数据的字典，或错误信息
55     """
56     try:
57         # 向 Frankfurter 汇率 API 发送 GET 请求（免费公开接口）
58         response = httpx.get(
59             f'https://api.frankfurter.app/{currency_date}',
60             params={'from': currency_from, 'to': currency_to},
61         )
62         # 如果 HTTP 请求失败（如 404、500），抛出异常
63         response.raise_for_status()
64
65         # 将响应解析为 JSON 格式
66         data = response.json()
67         # 检查返回数据是否包含 'rates' 字段（正常应有）
68         if 'rates' not in data:
69             return {'error': 'Invalid API response format.'}
70         # 返回完整的汇率数据（例如：{'date': '2025-11-24', 'rates': {'INR':
71         89.64}})
72         return data
73     except httpx.HTTPError as e:
74         # 捕获网络请求错误（如超时、连接失败）
75         return {'error': f'API request failed: {e}'}
76     except ValueError:
77         # 捕获 JSON 解析失败（比如返回了 HTML 错误页）
78         return {'error': 'Invalid JSON response from API.'}

```

```

79
80 # 定义 AI 最终输出的结构化格式
81 class ResponseFormat(BaseModel):
82     """规定 AI 回复必须包含状态和消息内容"""
83
84     # 状态只能是以下三种之一:
85     # - input_required: 需要用户补充信息
86     # - completed: 任务成功完成
87     # - error: 处理过程中出错
88     status: Literal['input_required', 'completed', 'error'] = 'input_required'
89     # 实际回复给用户的文字内容
90     message: str
91
92
93 # 定义一个专门处理货币兑换的智能体类
94 class CurrencyAgent:
95     """CurrencyAgent - 一个只负责查汇率的 AI 助手"""
96
97     # 系统指令: 告诉 AI 它的角色和限制
98     SYSTEM_INSTRUCTION = (
99         'You are a specialized assistant for currency conversions. '
100         "Your sole purpose is to use the 'get_exchange_rate' tool to answer "
101         "questions about currency exchange rates. "
102         'If the user asks about anything other than currency conversion or '
103         "exchange rates, "
104         'politely state that you cannot help with that topic and can only '
105         "assist with currency-related queries. "
106         'Do not attempt to answer unrelated questions or use tools for other '
107         "purposes."
108     )
109
110     # 格式指令: 指导 AI 如何设置最终响应的状态
111     FORMAT_INSTRUCTION = (
112         'Set response status to input_required if the user needs to provide '
113         "more information to complete the request."
114         'Set response status to error if there is an error while processing '
115         "the request."
116         'Set response status to completed if the request is complete.'
117     )
118
119     def __init__(self):
120         # 初始化大语言模型 (从环境变量读取配置)
121         self.model = ChatOpenAI(
122             model=os.getenv('TOOL_LLM_NAME'), # 模型名称 (如 'gpt-4' 或
123             # 本地模型名)
124             openai_api_key=os.getenv('API_KEY', 'EMPTY'), # API 密钥, 默认为
125             'EMPTY' (兼容本地模型)

```

```

118         openai_api_base=os.getenv('TOOL_LLM_URL'), # API 基础地址 (如本地
vLLM 或 Ollama 地址)
119         temperature=0, # 温度设为 0, 让回答更确定、不胡说
120     )
121     # 注册可用工具列表 (目前只有查汇率一个工具)
122     self.tools = [get_exchange_rate]
123
124     # 创建智能体: 结合模型、工具、记忆、系统提示和响应格式
125     self.graph = create_react_agent(
126         self.model, # 使用的语言模型
127         tools=self.tools, # 可调用的工具
128         checkpointer=memory, # 对话记忆存储
129         prompt=self.SYSTEM_INSTRUCTION, # 角色设定
130         response_format=(self.FORMAT_INSTRUCTION, ResponseFormat), # 要求最
终输出符合 ResponseFormat
131     )
132
133     # 异步流式响应方法: 边执行边返回中间状态
134     async def stream(self, query, context_id) -> AsyncIterable[dict[str, Any]]:
135         # 准备输入: 把用户问题包装成消息列表
136         inputs = {'messages': [('user', query)]}
137         # 配置会话 ID, 用于区分不同用户的对话历史
138         config = {'configurable': {'thread_id': context_id}}
139
140         # 让智能体逐步执行 (可能包括思考、调用工具、生成答案)
141         for item in self.graph.stream(inputs, config, stream_mode='values'):
142             # 获取当前步骤的最后一条消息
143             message = item['messages'][-1]
144             # 如果是 AI 决定要调用工具 (比如准备查汇率)
145             if (
146                 isinstance(message, AIMessage)
147                 and message.tool_calls
148                 and len(message.tool_calls) > 0
149             ):
150                 # 返回“正在查询”提示
151                 yield {
152                     'is_task_complete': False, # 任务未完成
153                     'require_user_input': False, # 不需要用户输入
154                     'content': 'Looking up the exchange rates...',
155                 }
156             # 如果是工具调用返回的结果 (比如拿到了汇率数据)
157             elif isinstance(message, ToolMessage):
158                 # 返回“正在处理”提示
159                 yield {
160                     'is_task_complete': False,
161                     'require_user_input': False,
162                     'content': 'Processing the exchange rates..',

```

```

163         }
164
165     # 所有步骤完成后, 获取并返回最终答案
166     yield self.get_agent_response(config)
167
168     # 获取智能体最终响应的方法
169     def get_agent_response(self, config):
170         # 从智能体状态中读取当前结果
171         current_state = self.graph.get_state(config)
172         # 尝试获取结构化响应 (即符合 ResponseFormat 的对象)
173         structured_response = current_state.values.get('structured_response')
174         # 如果确实拿到了规范格式的响应
175         if structured_response and isinstance(
176             structured_response, ResponseFormat
177         ):
178             # 如果状态是“需要更多信息”
179             if structured_response.status == 'input_required':
180                 return {
181                     'is_task_complete': False,
182                     'require_user_input': True,        # 需要用户补充信息
183                     'content': structured_response.message,
184                 }
185             # 如果状态是“出错”
186             if structured_response.status == 'error':
187                 return {
188                     'is_task_complete': False,
189                     'require_user_input': True,        # 提示用户可能需重试
190                     'content': structured_response.message,
191                 }
192             # 如果状态是“完成”
193             if structured_response.status == 'completed':
194                 return {
195                     'is_task_complete': True,          # 任务已完成
196                     'require_user_input': False,
197                     'content': structured_response.message,
198                 }
199
200         # 如果连结构化响应都没有 (异常兜底)
201         return {
202             'is_task_complete': False,
203             'require_user_input': True,
204             'content': (
205                 'We are unable to process your request at the moment. '
206                 'Please try again.'
207             ),
208         }
209

```

```
210 # 声明该智能体只支持纯文本输入/输出
211 SUPPORTED_CONTENT_TYPES = ['text', 'text/plain']
```

这个 `CurrencyAgent` 就像是银行里一个只会查汇率的智能客服机器人——你问它“10美元换多少人民币”，它会一步步告诉你“正在查…正在算…结果是72元”；如果你没说清楚换哪种货币，它会问你“请问换成什么币？”；如果你问天气，它会说“我只会查汇率哦”；如果网络断了，它也会诚实地告诉你“现在查不了，请稍后再试”。而且它还能记住你上一句说了什么，实现连续对话。

agent_executor.py

代码块

```
1  # 导入日志模块，用于记录程序运行信息（比如“开始处理请求”、“出错了”等）
2  import logging
3
4  # 从 a2a 框架导入核心执行组件：
5  # - AgentExecutor：所有智能体执行器的基类
6  # - RequestContext：封装当前用户请求的上下文（包含消息、任务、输入等）
7  from a2a.server.agent_execution import AgentExecutor, RequestContext
8
9  # 事件队列：用于将智能体的每一步操作（如回复消息、更新状态）推送给客户端
10 from a2a.server.events import EventQueue
11
12 # 任务更新器：专门用来更新任务状态（比如“进行中”、“等待输入”、“已完成”）
13 from a2a.server.tasks import TaskUpdater
14
15 # 从 a2a 类型系统导入各种标准错误和数据结构
16 from a2a.types import (
17     InternalError,          # 内部服务器错误（比如代码崩溃）
18     InvalidParamsError,    # 用户输入参数无效（比如格式不对）
19     Part,                  # 通用内容片段（可包含文本、图片等）
20     TaskState,             # 任务状态枚举（如 working, input_required,
    completed)
21     TextPart,              # 纯文本内容片段
22     UnsupportedOperationException, # 不支持的操作（比如取消任务）
23 )
24
25 # 工具函数：用于创建标准格式的消息和任务对象
26 from a2a.utils import (
27     new_agent_text_message, # 创建一条 AI 助手发送的文本消息
28     new_task,              # 创建一个新任务（当用户首次提问时）
29 )
30
31 # 错误包装类：将内部异常转换为符合 A2A 协议的错误响应
32 from a2a.utils.errors import ServerError
```

```

33
34 # 导入我们自己写的汇率智能体（前面定义的 CurrencyAgent）
35 from app.agent import CurrencyAgent
36
37
38 # 配置日志级别为 INFO，这样 info、warning、error 等日志都会打印出来
39 logging.basicConfig(level=logging.INFO)
40
41 # 获取当前模块的日志记录器，方便后续打日志
42 logger = logging.getLogger(__name__)
43
44
45 # 定义一个具体的智能体执行器，专门用于处理 CurrencyAgent 的请求
46 class CurrencyAgentExecutor(AgentExecutor):
47     """Currency Conversion AgentExecutor Example.
48
49     这个类负责把用户的请求交给 CurrencyAgent 处理，并按 A2A 协议规范返回结果。
50     """
51
52     def __init__(self):
53         # 初始化时创建一个 CurrencyAgent 实例（即那个只会查汇率的 AI）
54         self.agent = CurrencyAgent()
55
56     # 异步执行方法：这是 A2A 框架调用的核心入口
57     async def execute(
58         self,
59         context: RequestContext, # 包含用户输入、当前任务等信息
60         event_queue: EventQueue, # 用于向客户端推送事件（如消息、状态更新）
61     ) -> None:
62         # 第一步：验证用户请求是否合法
63         error = self._validate_request(context)
64         if error:
65             # 如果验证失败，抛出“参数无效”错误（但目前 _validate_request 总是返回
66             # False，所以这行不会触发）
67             raise ServerError(error=InvalidParamsError())
68
69         # 从上下文中提取用户输入的文本（比如“10美元换多少人民币？”）
70         query = context.get_user_input()
71
72         # 获取当前任务（一次对话可能对应一个任务）
73         task = context.current_task
74
75         # 如果没有任务（比如用户第一次提问），就创建一个新任务
76         if not task:
77             task = new_task(context.message) # type: ignore
78             # 把新任务事件推送给客户端（告诉客户端：“有个新任务开始了”）
79             await event_queue.enqueue_event(task)

```

```

79     # 创建任务更新器，用于后续更新该任务的状态和消息
80     updater = TaskUpdater(event_queue, task.id, task.context_id)
81
82
83     try:
84         # 让 CurrencyAgent 流式处理用户问题，并逐步返回中间结果
85         async for item in self.agent.stream(query, task.context_id):
86             # 从流式结果中提取关键字段
87             is_task_complete = item['is_task_complete']      # 任务是否最终完
成？
88             require_user_input = item['require_user_input'] # 是否需要用户补
充信息？
89
90             # 情况1：任务还在进行中，且不需要用户输入（比如正在查汇率）
91             if not is_task_complete and not require_user_input:
92                 # 更新任务状态为“working”（工作中），并发送一条提示消息（如“正在查
询...”）
93                 await updater.update_status(
94                     TaskState.working,
95                     new_agent_text_message(
96                         item['content'],      # 消息内容
97                         task.context_id,      # 会话 ID
98                         task.id,              # 任务 ID
99                     ),
100                 )
101
102             # 情况2：需要用户补充信息（比如没说换成哪种货币）
103             elif require_user_input:
104                 # 更新任务状态为“input_required”（等待输入），并发送提示消息
105                 await updater.update_status(
106                     TaskState.input_required,
107                     new_agent_text_message(
108                         item['content'],
109                         task.context_id,
110                         task.id,
111                     ),
112                     final=True, # 标记这是本轮交互的最终状态（等待用户回复）
113                 )
114                 break # 停止处理，等待用户下一次输入
115
116             # 情况3：任务成功完成（比如已算出“10美元=72元人民币”）
117             else:
118                 # 将最终结果作为“工件”（artifact）保存（便于后续引用或展示）
119                 await updater.add_artifact(
120                     [Part(root=TextPart(text=item['content']))], # 包装成标
准文本片段
121                     name='conversion_result', # 工件名称

```

```

122         )
123         # 标记任务为“completed” (已完成)
124         await updater.complete()
125         break # 结束处理
126
127     # 如果处理过程中发生任何异常 (比如网络错误、代码 bug)
128     except Exception as e:
129         # 记录错误日志, 方便排查问题
130         logger.error(f'An error occurred while streaming the response:
131 {e}')
132         # 向客户端返回“内部服务器错误”
133         raise ServerError(error=InternalServerError()) from e
134
135     # 请求验证方法 (目前未实现有效逻辑)
136     def _validate_request(self, context: RequestContext) -> bool:
137         # 返回 False 表示“没有错误”, 即总是通过验证
138         # (如果未来想检查输入长度、格式等, 可以在这里加逻辑)
139         return False
140
141     # 取消任务的方法 (目前不支持)
142     async def cancel(
143         self, context: RequestContext, event_queue: EventQueue
144     ) -> None:
145         # 直接抛出“不支持的操作”错误 (因为这个智能体无法中途取消)
146         raise ServerError(error=UnsupportedOperationError())

```

想象你在使用一个银行官方 App 的汇率查询功能：

- 你输入：“10美元能换多少人民币？”
 - App 显示：“正在查询汇率…” → “正在计算…” → 最后弹出：“10美元 ≈ 72元人民币”

 - 这就是 `execute` 方法在做的事：调用 `CurrencyAgent`，一步步把状态和结果推送给 App。
- 你只输入：“1美元等于多少？”
 - App 回复：“请问您要换成哪种货币？例如欧元、日元或人民币。” 
 - 这时任务状态变成 `input_required`，App 会等待你补充“人民币”后再继续。
- 如果你突然点“取消”，App 提示：“当前操作不支持取消。” 
 - 因为 `cancel` 方法直接报错，表示这个功能还没实现（或没必要实现）。
- 如果银行的汇率服务器宕机了，App 会显示：“服务暂时不可用，请稍后再试。” 
 - 这就是 `except Exception` 捕获错误后返回 `InternalServerError` 的效果。

总结： `CurrencyAgentExecutor` 就像是连接用户 App 和后台汇率 AI 的“调度员”。它接收用户请求，指挥 `CurrencyAgent` 工作，并严格按照 A2A 协议把每一步进展（查汇率、等输入、出结果、报错）实时推送给前端，确保用户体验流畅、规范。

`__main__.py`

代码块

```
1  # 导入日志模块，用于记录程序运行状态（如启动成功、缺少配置等）
2  import logging
3
4  # 导入操作系统接口，用于读取环境变量（如 API 地址、模型名等）
5  import os
6
7  # 导入系统模块，用于在出错时安全退出程序
8  import sys
9
10 # 导入 Click 库，用于创建命令行界面（比如通过 `--host` 和 `--port` 设置参数）
11 import click
12
13 # 导入 httpx 异步 HTTP 客户端，用于发送网络请求（比如推送通知）
14 import httpx
15
16 # 导入 Uvicorn，一个高性能 ASGI 服务器，用于运行 FastAPI/Starlette 应用
17 import uvicorn
18
19 # 从 a2a 框架导入核心应用类：A2AStarletteApplication 是符合 A2A 协议的 Web 服务入口
20 from a2a.server.apps import A2AStarletteApplication
21
22 # 默认请求处理器：负责接收用户请求并交给智能体执行器处理
23 from a2a.server.request_handlers import DefaultRequestHandler
24
25 # 推送通知和任务存储相关组件（内存版，适合开发测试）
26 from a2a.server.tasks import (
27     BasePushNotificationSender,          # 用于向客户端推送实时消息
28     InMemoryPushNotificationConfigStore,  # 内存中保存用户的推送配置（如
                                           WebSocket 连接）
29     InMemoryTaskStore,                  # 内存中保存任务状态（如“进行中”、“已
                                           完成”）
30 )
31
32 # A2A 协议定义的标准数据结构：描述智能体的能力、技能卡片等
33 from a2a.types import (
34     AgentCapabilities,  # 智能体支持的功能（如是否支持流式响应、推送通知）
35     AgentCard,          # 智能体的“名片”，包含名称、描述、技能等信息
36     AgentSkill,         # 智能体的一项具体能力（如“查汇率”）
37 )
38
```

```

39 # 从 dotenv 加载 .env 文件中的环境变量 (方便本地开发配置)
40 from dotenv import load_dotenv
41
42 # 导入我们自己实现的汇率智能体和它的执行器
43 from app.agent import CurrencyAgent
44 from app.agent_executor import CurrencyAgentExecutor
45
46
47 # 自动加载项目根目录下的 .env 文件中的环境变量 (如 TOOL_LLM_URL=...)
48 load_dotenv()
49
50 # 配置日志输出级别为 INFO, 这样 info、warning、error 都会显示
51 logging.basicConfig(level=logging.INFO)
52
53 # 获取当前模块的日志记录器, 用于打印带模块名的日志
54 logger = logging.getLogger(__name__)
55
56
57 # 自定义异常: 当缺少必要环境变量时抛出
58 class MissingAPIKeyError(Exception):
59     """Exception for missing API key."""
60     pass # 只是一个标记类, 表示“缺少关键配置”
61
62
63 # 定义一个命令行命令 (通过 `click` 实现)
64 @click.command()
65 # 允许用户通过 --host 指定监听的主机地址, 默认是 'localhost'
66 @click.option('--host', 'host', default='localhost')
67 # 允许用户通过 --port 指定监听的端口号, 默认是 10001
68 @click.option('--port', 'port', default=10001)
69 def main(host, port):
70     """Starts the Currency Agent server."""
71     try:
72         # 检查是否设置了大模型的 API 地址 (比如本地 LLM 服务的 URL)
73         if not os.getenv('TOOL_LLM_URL'):
74             raise MissingAPIKeyError(
75                 'TOOL_LLM_URL environment variable not set.'
76             )
77         # 检查是否设置了要使用的模型名称 (如 'gpt-4' 或 'qwen-max')
78         if not os.getenv('TOOL_LLM_NAME'):
79             raise MissingAPIKeyError(
80                 'TOOL_LLM_NAME environment variable not set.' # 修复原文拼写错误: 原为 "environment not variable"
81             )
82
83         # 声明该智能体支持的能力:
84         # - streaming=True: 支持边思考边回复 (流式输出)

```

```

85         # - push_notifications=True: 支持主动推送消息给客户端（如任务更新）
86         capabilities = AgentCapabilities(streaming=True,
push_notifications=True)
87
88         # 定义智能体的一项具体技能：“查汇率”
89         skill = AgentSkill(
90             id='convert_currency', # 技能唯一标识
91             name='Currency Exchange Rates Tool', # 技能名称
92             description='Helps with exchange values between various
currencies', # 描述
93             tags=['currency conversion', 'currency exchange'], # 标签，便于分类
94             examples=['What is exchange rate between USD and GBP?'], # 使用示例
95         )
96
97         # 创建智能体的“电子名片”（AgentCard），供客户端发现和调用
98         agent_card = AgentCard(
99             name='Currency Agent', # 名称
100             description='Helps with exchange rates for currencies', # 简介
101             url=f'http://{host}:{port}/', # 服务地址
102             version='1.0.0', # 版本号
103             default_input_modes=CurrencyAgent.SUPPORTED_CONTENT_TYPES, # 支持的
输入类型（纯文本）
104             default_output_modes=CurrencyAgent.SUPPORTED_CONTENT_TYPES, # 支持的
输出类型（纯文本）
105             capabilities=capabilities, # 支持的能力
106             skills=[skill], # 提供的技能列表
107         )
108
109         # --8<-- [start:DefaultRequestHandler]
110         # 创建一个异步 HTTP 客户端，用于后续发送推送通知（如通过 Webhook）
111         httpx_client = httpx.AsyncClient()
112
113         # 创建内存版的推送配置存储（保存每个用户的推送设置，如回调 URL）
114         push_config_store = InMemoryPushNotificationConfigStore()
115
116         # 创建推送通知发送器，绑定 HTTP 客户端和配置存储
117         push_sender = BasePushNotificationSender(
118             httpx_client=httpx_client,
119             config_store=push_config_store
120         )
121
122         # 创建默认请求处理器：
123         # - 使用我们自定义的 CurrencyAgentExecutor 来执行任务
124         # - 使用内存任务存储（InMemoryTaskStore）记录任务状态
125         # - 绑定推送配置和发送器，支持实时通知
126         request_handler = DefaultRequestHandler(
127             agent_executor=CurrencyAgentExecutor(),

```

```

128         task_store=InMemoryTaskStore(),
129         push_config_store=push_config_store,
130         push_sender=push_sender
131     )
132
133     # 创建 A2A 协议兼容的 Web 应用
134     server = A2AStarletteApplication(
135         agent_card=agent_card,          # 智能体名片
136         http_handler=request_handler # 请求处理逻辑
137     )
138
139     # 启动 Web 服务：使用 Uvicorn 运行 Starlette 应用
140     uvicorn.run(server.build(), host=host, port=port)
141     # --8<-- [end:DefaultRequestHandler]
142
143     # 如果缺少必要环境变量，记录错误并退出
144     except MissingAPIKeyError as e:
145         logger.error(f'Error: {e}')
146         sys.exit(1) # 退出码 1 表示“配置错误”
147
148     # 如果发生其他未预期的异常（如端口被占用、依赖缺失等）
149     except Exception as e:
150         logger.error(f'An error occurred during server startup: {e}')
151         sys.exit(1) # 退出码 1 表示“启动失败”
152
153
154     # 当直接运行此脚本时（而不是被导入），执行 main 函数
155     if __name__ == '__main__':
156         main()

```

test_client.py

代码块

```

1  # 导入日志模块，用于在运行时打印提示、警告或错误信息，方便调试和观察程序行为
2  import logging
3
4  # 导入通用类型 Any，表示变量可以是任意类型（灵活但需谨慎使用）
5  from typing import Any
6
7  # 导入 uuid4 函数，用于生成全球唯一的随机 ID（比如消息ID、请求ID），避免重复
8  from uuid import uuid4
9
10 # 导入 httpx 异步 HTTP 客户端库，用于向远程服务（如智能体服务器）发送网络请求
11 import httpx
12

```

```

13 # 从 a2a 客户端库导入两个核心工具类：
14 # - A2ACardResolver: 用于自动获取智能体的“能力说明书” (Agent Card)
15 # - A2AClient: 用于向智能体发送消息并接收回复
16 from a2a.client import A2ACardResolver, A2AClient
17
18 # 从 a2a 类型定义模块导入所需的数据结构：
19 # - AgentCard: 智能体的能力描述文件 (类似电子名片)
20 # - MessageSendParams: 发送消息时的参数格式
21 # - SendMessageRequest: 一次完整的消息请求封装 (用于普通回复)
22 # - SendStreamingMessageRequest: 一次流式消息请求封装 (用于逐步返回结果)
23 from a2a.types import (
24     AgentCard,
25     MessageSendParams,
26     SendMessageRequest,
27     SendStreamingMessageRequest,
28 )
29
30 # 从常量模块导入 A2A 协议规定的标准路径：
31 # - AGENT_CARD_WELL_KNOWN_PATH: 公开智能体名片的标准访问路径 (通常是 /.well-known/agent-card.json)
32 # - EXTENDED_AGENT_CARD_PATH: 需要认证的扩展名片路径 (通常是 /extended-agent-card)
33 from a2a.utils.constants import (
34     AGENT_CARD_WELL_KNOWN_PATH,
35     EXTENDED_AGENT_CARD_PATH,
36 )
37
38
39 # 定义主异步函数 main, 整个客户端测试流程都在这里执行
40 async def main() -> None:
41     # 配置日志系统, 设置最低显示级别为 INFO (会显示 info、warning、error 等日志)
42     logging.basicConfig(level=logging.INFO)
43     # 获取当前模块的日志记录器, 方便后续打带模块名的日志
44     logger = logging.getLogger(__name__)
45
46     # --8<-- [start:A2ACardResolver]
47
48     # 指定智能体服务运行的地址 (本地 10001 端口, 与服务端启动端口一致)
49     base_url = 'http://localhost:10001'
50
51     # 创建一个异步 HTTP 客户端, 设置总超时和读取超时均为 60 秒, 防止请求卡死
52     async with httpx.AsyncClient(timeout=httpx.Timeout(60.0, read=60.0)) as
httpx_client:
53         # 初始化“智能体名片解析器”, 它能帮我们自动从服务器拉取智能体的能力说明
54         resolver = A2ACardResolver(
55             httpx_client=httpx_client, # 使用上面创建的 HTTP 客户端
56             base_url=base_url,         # 智能体服务的基础 URL
57             # agent_card_path 和 extended_agent_card_path 使用默认值 (即标准路径)

```

```

58         )
59         # --8<-- [end:A2ACardResolver]
60
61         # 声明一个变量，用于保存最终要使用的智能体名片（可能是公开版或扩展版）
62         final_agent_card_to_use: AgentCard | None = None
63
64         try:
65             # 打印日志：准备从标准公开路径获取智能体名片
66             logger.info(
67                 f'Attempting to fetch public agent card from: {base_url}
{AGENT_CARD_WELL_KNOWN_PATH}'
68             )
69             # 调用解析器获取公开名片（默认访问 /.well-known/agent-card.json）
70             _public_card = (
71                 await resolver.get_agent_card()
72             ) # 从默认公开路径拉取
73             # 打印成功获取的日志
74             logger.info('Successfully fetched public agent card:')
75             # 将名片内容以格式化 JSON 形式打印出来（去掉空字段，缩进2格，便于阅读）
76             logger.info(
77                 _public_card.model_dump_json(indent=2, exclude_none=True)
78             )
79             # 默认先使用公开名片
80             final_agent_card_to_use = _public_card
81             logger.info(
82                 '\nUsing PUBLIC agent card for client initialization
(default).'
83             )
84
85             # 检查公开名片是否声明“支持认证后的扩展名片”
86             if _public_card.supports_authenticated_extended_card:
87                 try:
88                     # 如果支持，就尝试用 token 获取更详细的扩展名片
89                     logger.info(
90                         '\nPublic card supports authenticated extended card. '
91                         'Attempting to fetch from: '
92                         f'{base_url}{EXTENDED_AGENT_CARD_PATH}'
93                     )
94                     # 构造一个模拟的认证请求头（实际项目中应使用真实 token）
95                     auth_headers_dict = {
96                         'Authorization': 'Bearer dummy-token-for-extended-card'
97                     }
98                     # 再次调用解析器，但这次指定路径为扩展名片路径，并带上认证头
99                     _extended_card = await resolver.get_agent_card(
100                         relative_card_path=EXTENDED_AGENT_CARD_PATH, # 指定非默

```

认路径

```

101             http_kwargs={'headers': auth_headers_dict},      # 附加
    HTTP 请求参数
102         )
103         # 打印成功获取扩展名片的日志
104         logger.info(
105             'Successfully fetched authenticated extended agent
card:'
106         )
107         # 格式化打印扩展名片内容
108         logger.info(
109             _extended_card.model_dump_json(
110                 indent=2, exclude_none=True
111             )
112         )
113         # 更新为使用扩展名片（因为它可能包含更多私有技能）
114         final_agent_card_to_use = (
115             _extended_card
116         )
117         logger.info(
118             '\nUsing AUTHENTICATED EXTENDED agent card for client '
119             'initialization.'
120         )
121     except Exception as e_extended:
122         # 如果获取扩展名片失败，记录警告，但不中断流程，继续用公开名片
123         logger.warning(
124             f'Failed to fetch extended agent card: {e_extended}. '
125             'Will proceed with public card.',
126             exc_info=True, # 同时打印完整的错误堆栈，便于排查
127         )
128     elif (
129         _public_card
130     ): # 如果公开名片存在，但明确不支持或未声明支持扩展名片
131         logger.info(
132             '\nPublic card does not indicate support for an extended
card. Using public card.'
133         )
134
135     except Exception as e:
136         # 如果连公开名片都拿不到，说明服务没启动或网络不通，属于严重错误
137         logger.error(
138             f'Critical error fetching public agent card: {e}',
139             exc_info=True
140         )
141         # 抛出运行时错误，终止程序
142         raise RuntimeError(
143             'Failed to fetch the public agent card. Cannot continue.'
144         ) from e

```

```

144
145     # --8<-- [start:send_message]
146     # 使用最终确定的智能体名片初始化 A2A 客户端（它知道如何与该智能体对话）
147     client = A2AClient(
148         httpx_client=httpx_client, agent_card=final_agent_card_to_use
149     )
150
151     # （以下两行是另一种创建客户端的方式，目前被注释掉，未启用）
152     # from a2a.client import ClientFactory
153     # client = ClientFactory.create_client(agent_card=card, timeout=60)
154
155     # 记录客户端初始化成功的日志
156     logger.info('A2AClient initialized.')
157
158     # 构造第一条用户消息的内容：询问“10美元换多少印度卢比？”
159     send_message_payload: dict[str, Any] = {
160         'message': {
161             'role': 'user',                # 角色是用户（不是系统或助手）
162             'parts': [                    # 消息由多个“片段”组成（这里只
有一个文本片段）
163                 {'kind': 'text', 'text': 'how much is 10 USD in INR?'}
164             ],
165             'message_id': uuid4().hex,    # 生成唯一消息ID（十六进制字符
串，确保不重复）
166         },
167     }
168     # 将消息包装成符合 A2A 协议的完整请求对象
169     request = SendMessageRequest(
170         id=str(uuid4()),                # 整个请求的全局唯一ID
171         params=MessageSendParams(**send_message_payload) # 填入消息参数
172     )
173
174     # 发送消息并等待服务器返回完整响应（非流式，一次性返回）
175     response = await client.send_message(request)
176     # 打印服务器返回的 JSON 结果（去掉空字段，便于阅读）
177     print(response.model_dump(mode='json', exclude_none=True))
178     # --8<-- [end:send_message]
179
180     # --8<-- [start:Multiturn]
181     # 构造一个多轮对话的第一条消息：只问“1美元汇率是多少？”，但没说换成什么币
182     send_message_payload_multiturn: dict[str, Any] = {
183         'message': {
184             'role': 'user',
185             'parts': [
186                 {
187                     'kind': 'text',
188                     'text': 'How much is the exchange rate for 1 USD?',

```

```

189         }
190     ],
191     'message_id': uuid4().hex,
192 },
193 }
194 # 包装成请求对象
195 request = SendMessageRequest(
196     id=str(uuid4()),
197     params=MessageSendParams(**send_message_payload_multiturn),
198 )
199
200 # 发送第一轮消息
201 response = await client.send_message(request)
202 # 打印响应
203 print(response.model_dump(mode='json', exclude_none=True))
204
205 # 从响应中提取任务ID和上下文ID (用于关联后续对话, 实现“记住上下文”)
206 task_id = response.root.result.id
207 context_id = response.root.result.context_id
208
209 # 构造第二轮消息: 用户补充“换成加元 (CAD) ”
210 second_send_message_payload_multiturn: dict[str, Any] = {
211     'message': {
212         'role': 'user',
213         'parts': [{'kind': 'text', 'text': 'CAD'}],
214         'message_id': uuid4().hex,
215         'task_id': task_id,          # 关键: 带上上一轮的任务ID
216         'context_id': context_id,   # 关键: 带上上一轮的上下文ID
217     },
218 }
219
220 # 包装第二轮请求
221 second_request = SendMessageRequest(
222     id=str(uuid4()),
223     params=MessageSendParams(**second_send_message_payload_multiturn),
224 )
225
226 # 发送第二轮消息
227 second_response = await client.send_message(second_request)
228 # 打印第二轮响应
229 print(second_response.model_dump(mode='json', exclude_none=True))
230 # --8<-- [end:Multiturn]
231
232 # --8<-- [start:send_message_streaming]
233 # 再次发送最初的问题, 但这次要求“流式回复” (像真人打字一样逐步输出)
234 streaming_request = SendStreamingMessageRequest(
235     id=str(uuid4()),

```

```

236         params=MessageSendParams(**send_message_payload) # 复用第一条消息内
容
237     )
238
239     # 调用流式发送方法，返回一个异步可迭代对象（数据流）
240     stream_response = client.send_message_streaming(streaming_request)
241
242     # 逐块接收服务器返回的内容（每收到一块就打印一次）
243     async for chunk in stream_response:
244         print(chunk.model_dump(mode='json', exclude_none=True))
245     # --8<-- [end:send_message_streaming]
246
247
248 # 如果这个文件被直接运行（而不是被其他文件导入），就启动异步主函数
249 if __name__ == '__main__':
250     import asyncio # 导入异步事件循环管理模块
251
252     # 启动 asyncio 事件循环，运行 main 函数
253     asyncio.run(main())

```

🌟 生活中的例子说明

想象你正在使用一个智能客服机器人（比如银行的汇率助手），而这段代码就是你在后台测试这个机器人是否正常工作的“遥控器”。

1. 第一步：看说明书

你先问机器人：“你能干什么？”

→ 它给你一份《公开说明书》：“我会查汇率。”

→ 你还试着用员工卡刷一下，看能不能拿到《内部高级说明书》，结果发现不需要，就继续用公开版。

2. 第二步：单次提问

你问：“10美元换多少印度卢比？”

→ 机器人思考后，直接告诉你答案 

3. 第三步：多轮对话

你故意只说：“1美元汇率多少？”

→ 机器人回：“请问换成哪种货币？”

→ 你接着说：“加元。”

→ 机器人立刻回答：“1美元 $\approx$ 1.37 加元。”

→ 关键：它记得这是同一个问题，没有当成两个独立问题！

4. 第四步：边想边说（流式）

你再次提问，但要求“边查边说”。

→ 屏幕上依次出现：

“正在查询…” → “获取到最新汇率…” → “10美元 = 835.20 INR”

→ 就像真人打字一样，让你感觉它在“实时思考”。

总结：这段代码是一个完整的 A2A 智能体客户端测试脚本。它模拟了真实用户与智能体交互的所有典型场景——获取能力说明、单轮问答、多轮上下文对话、流式响应——确保你开发的智能体服务符合 A2A 协议规范，能被任何兼容的前端（App、网页、语音助手）无缝调用。